Міністерство освіти і науки України
Національний університет «Полтавська політехніка імені Юрія Кондратюка»
Навчально-науковий інститут інформаційних технологій та робототехніки
Кафедра комп'ютерних та інформаційних технологій і систем

ЗВІТ

з лабораторної роботи №5
з дисципліни «Технології на платформі .NET»

на тему: «Реєстрація, автентифікація та авторизація засобами ASP.NET Core Identity; розгортання в хмарі»

Виконав: студент групи 402-ТН
Желудков Віталій Віталійович

Перевірив: Тютюнник П. Б.

Полтава — 2026

1. Опис виконаної роботи

Роботу виконано в гілці `lab5`, створеній від гілки `lab4`. Пул реквест лабораторної роботи №4 ще не закрито, тому PR створено з гілки `lab5` у `lab4`. До REST API з ЛР4 додано реєстрацію, автентифікацію за токенами Bearer і авторизацію за ролями засобами ASP.NET Core Identity. Застосунок розгорнуто на хмарній платформі Render (додаткове завдання).

Розгорнутий застосунок (Swagger UI): https://gamestudio-api-ni3x.onrender.com/swagger

Гілка lab5:
https://github.com/vitalikcraft666-design/NUPP_NET_2026_402_TK_Zheludkov_Lab/tree/lab5

Репозиторій:
https://github.com/vitalikcraft666-design/NUPP_NET_2026_402_TK_Zheludkov_Lab

Змінені та нові файли:

Проєкт	Що додано або змінено
`GameStudio.Infrastructure`	пакет `Microsoft.AspNetCore.Identity.EntityFrameworkCore` 8.0.32; папка `Identity`: користувач `StudioUser`, менеджер `StudioUserManager`, `IdentitySeeder` (ролі й адміністратор); `GameStudioContext` тепер успадковує `IdentityDbContext<StudioUser>`; конфігурація `StudioUserConfiguration`; міграція `AddIdentity`
`GameStudio.Common`	`Security/StudioRoles.cs` — назви ролей і їх комбінації для атрибутів авторизації
`GameStudio.REST`	налаштування Identity, токенів Bearer і Swagger (`Program.cs`); вбудовані кінцеві точки `/api/auth/*`; атрибути `[Authorize]` / `[AllowAnonymous]` у контролерах ЛР4; нові контролери `AccountController` і `UsersController`; фільтр Swagger `AuthorizationOperationFilter`
`GameStudio.Tests`	базовий клас `ApiTestBase` (клієнти з токенами), 11 тестів `AuthTests`
корінь репозиторію	`Dockerfile`, `.dockerignore`, `render.yaml` — розгортання на Render

1.1. Сутність користувача StudioUser

До проєкту `GameStudio.Infrastructure` додано NuGet-пакет `Microsoft.AspNetCore.Identity.EntityFrameworkCore`. Сутність користувача успадковує `IdentityUser` і додає два власні поля:

```
public class StudioUser : IdentityUser
{
    public string? FullName { get; set; }       // ПІБ, до 100 символів (Fluent API)
    public DateTime RegisteredAt { get; set; }  // дата реєстрації (UTC)
}
```

Від `IdentityUser` успадковано `Id` (рядок з Guid), `UserName`, `Email`, `PasswordHash`, `SecurityStamp`, `LockoutEnd`, `AccessFailedCount` та інші стандартні властивості. Поле `RegisteredAt` і роль за замовчуванням задає `StudioUserManager` — нащадок `UserManager<StudioUser>`, у якому перевизначено `CreateAsync`. Тому вбудована кінцева точка `/register` без жодних змін створює користувача з роллю **Tester**.

1.2. GameStudioContext успадковує IdentityDbContext<StudioUser>

```
public class GameStudioContext : IdentityDbContext<StudioUser>
{
    // DbSet-и ЛР3: Employees, Programmers, Artists, GameProjects, StorePages, BugReports,
ProjectAssignments
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);   // таблиці, ключі та індекси Identity
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(GameStudioContext).Assembly);
    }
}
```

Зміни схеми БД оформлено міграцією `AddIdentity`. Вона додає сім таблиць Identity і не змінює таблиць ЛР3: `AspNetUsers` (зі стовпцями `FullName` і `RegisteredAt`), `AspNetRoles`, `AspNetUserRoles`, `AspNetUserClaims`, `AspNetRoleClaims`, `AspNetUserLogins`, `AspNetUserTokens`. Міграцію створено й застосовано до файлу БД з ЛР3:

```
$ dotnet ef migrations add AddIdentity --project GameStudio.Infrastructure
Build started...
Build succeeded.
Done. To undo this action, use 'ef migrations remove'
$ dotnet ef migrations list --project GameStudio.Infrastructure
Build started...
Build succeeded.
20261021213932_InitialCreate
20261105095327_AddIdentity (Pending)
$ dotnet ef database update --project GameStudio.Infrastructure
Build started...
Build succeeded.
Applying migration '20261105095327_AddIdentity'.
Done.
```

Під час запуску застосунок викликає `Database.MigrateAsync()`, а `IdentitySeeder` через `RoleManager` створює чотири ролі та через `UserManager` — адміністратора. Його email і пароль беруться з конфігурації (`Seed:AdminEmail`, `Seed:AdminPassword`). Пароль у репозиторії не зберігається: у `appsettings.json` він порожній, а на сервері його задано змінною середовища.

1.3. Реєстрація, вхід і токени Bearer

У `Program.cs` проєкту `GameStudio.REST` підключено ASP.NET Core Identity з вбудованими кінцевими точками (за офіційним посібником Microsoft «Use Identity to secure a Web API backend for SPAs»):

```
builder.Services
    .AddIdentityApiEndpoints<StudioUser>(options =>
    {
        options.User.RequireUniqueEmail = true;
        options.Password.RequiredLength = 8;
        options.Lockout.MaxFailedAccessAttempts = 5;
    })
    .AddRoles<IdentityRole>()                          // RoleManager<IdentityRole> і ролі в токені
    .AddUserManager<StudioUserManager>()               // роль Tester для кожного нового користувача
    .AddEntityFrameworkStores<GameStudioContext>();    // користувачі та ролі зберігаються в БД з ЛР3

// час життя токена доступу; токен оновлення (refreshToken) дійсний 14 днів
```

```
builder.Services.Configure<BearerTokenOptions>(IdentityConstants.BearerScheme,
    options => options.BearerTokenExpiration = TimeSpan.FromHours(1));

builder.Services.AddAuthorization();
```

Кінцеві точки відкрито групою `app.MapGroup("/api/auth").WithTags("Auth").MapIdentityApi<StudioUser>()`:

Кінцева точка	Призначення
`POST /api/auth/register`	реєстрація: `{ email, password }` → 200 або 400 з описом помилок (слабкий пароль, email уже зайнятий)
`POST /api/auth/login`	вхід: `{ email, password }` → 200 `{ tokenType: "Bearer", accessToken, expiresIn: 3600, refreshToken }` або 401
`POST /api/auth/refresh`	новий токен доступу за `refreshToken` (у ньому вже враховано змінені ролі)
`GET/POST /api/auth/manage/info`	email користувача, зміна email чи пароля
`/confirmEmail`, `/forgotPassword`, `/resetPassword`, `/manage/2fa`	інші вбудовані операції Identity (підтвердження пошти, скидання пароля, двофакторна автентифікація)

Токен доступу (`accessToken`) діє 1 годину, токен оновлення — 14 днів. Це токени Bearer ASP.NET Core Identity: дані користувача (Id, ім'я, email, ролі, `SecurityStamp`) зашифровано й підписано ключами Data Protection сервера. Клієнт передає токен у заголовку `Authorization: Bearer <accessToken>`. Пароль має містити щонайменше 8 символів, велику й малу літери, цифру та спецсимвол. Після 5 невдалих спроб входу обліковий запис блокується на 5 хвилин.

1.4. Ролі та розподіл доступу

Створено чотири ролі (вимога — не менше трьох). Права зростають від Tester до Admin:

Роль	Хто це	Що дозволено
Tester	кожен зареєстрований користувач	повідомляти про баги (`POST /api/bugs`), переглядати й редагувати власний профіль
Developer	розробник	+ оновлювати баги (виконавець, стан «виправлено»), переглядати працівників
Manager	продюсер	+ створювати й змінювати проєкти, сторінки в магазині, склад команди; видаляти баги
Admin	адміністратор	+ видаляти проєкти; приймати, змінювати й звільняти працівників; керувати ролями користувачів

На кожному контролері стоїть `[Authorize]`, тому за замовчуванням доступ закрито. Відкриті методи позначено `[AllowAnonymous]`, а методи для окремих ролей — `[Authorize(Roles = StudioRoles.AdminOrManager)]` тощо. Константи ролей (`Admin`, `Manager`, `Developer`, `Tester`, `AdminOrManager = "Admin,Manager"`, `Staff = "Admin,Manager,Developer"`) зібрано в класі `StudioRoles`. Повна матриця доступу:

Кінцеві точки	Без входу	Tester	Developer	Manager	Admin
GET проєкти, сторінка в магазині, баги проєкту, команда; GET баги	✔	✔	✔	✔	✔
POST /api/bugs — повідомити про баг	401	✔	✔	✔	✔
GET, PUT /api/account/me — власний профіль	401	✔	✔	✔	✔
PUT /api/bugs/{id} — оновити баг	401	403	✔	✔	✔
GET програмісти, художники	401	403	✔	✔	✔
POST, PUT проєкти; PUT/DELETE сторінка в магазині і команда; DELETE баг	401	403	403	✔	✔
DELETE /api/projects/{id}	401	403	403	403	✔
POST, PUT, DELETE програмісти, художники	401	403	403	403	✔
/api/users — користувачі та ролі	401	403	403	403	✔

Нові контролери: `AccountController` (`GET` і `PUT /api/account/me` — профіль поточного користувача, визначеного за токеном) і `UsersController` (лише Admin): список користувачів, список ролей, надання (`PUT /api/users/{id}/roles/{role}`) і відкликання (`DELETE …`) ролі. Адміністратор не може забрати роль Admin сам у себе (409).

1.5. Відповіді 401 і 403, Swagger

Якщо токена немає або він недійсний, обробник Bearer повертає **401 Unauthorized** із заголовком `WWW-Authenticate: Bearer`. Якщо користувача впізнано, але його роль не дозволяє дію, повертається **403 Forbidden**. Ці відповіді не мають тіла, тому `UseStatusCodePages()` разом з `AddProblemDetails` додає до них тіло ProblemDetails (`application/problem+json`) з поясненням українською.

На сторінці Swagger є кнопка **Authorize** (схема безпеки `Bearer`). Фільтр `AuthorizationOperationFilter` позначає захищені операції замком, додає до них відповіді 401 / 403 і перелік ролей, яким дозволено доступ (наприклад, «Доступ: ролі Admin, Manager.»).

1.6. Інтеграційні тести

Додано 11 тестів `AuthTests`: реєстрація та вхід, слабкий пароль і повторний email, неправильний пароль, роль Tester за замовчуванням, 401 для анонімного клієнта й недійсного токена, 403 для недостатньої ролі, можливості кожної ролі, керування ролями і оновлення токена. Тести ЛР4 тепер виконуються від імені адміністратора. Усього 72 тестів, усі пройшли (розділ 4).

2. Додаткове завдання: розгортання на Render

Застосунок розгорнуто на хмарній платформі **Render** (render.com) як вебсервіс (Web Service) у Docker-контейнері. Адреса застосунку: **https://gamestudio-api-ni3x.onrender.com** (корінь перенаправляє на Swagger UI `/swagger`).

2.1. Інфраструктура

Параметр	Значення
Сервіс	Render Web Service `gamestudio-api` (ID `srv-dbm5fn8ae00c738dovl0`), безкоштовний план Free
Регіон	Frankfurt (EU Central)
Джерело коду	публічний репозиторій GitHub, гілка `lab5`; автоматичне розгортання після кожного push
Збирання	`Dockerfile` у корені репозиторію: образи `mcr.microsoft.com/dotnet/sdk:8.0` (збирання) і `aspnet:8.0` (запуск)
HTTPS і домен	сертифікат TLS і домен `*.onrender.com` надає Render; HTTPS завершується на проксі-сервері, застосунок отримує HTTP на порту `PORT` (10000)
Перевірка стану	`/health` (ASP.NET Core Health Checks): Render вважає розгортання успішним, коли ця адреса повертає 200
База даних	SQLite з ЛР3 (`Database/gamestudio.db`) усередині контейнера; міграція `AddIdentity`, ролі й адміністратор застосовуються під час запуску
Змінні середовища	`Seed__AdminEmail`, `Seed__AdminPassword` (секрет); у Dockerfile — `ASPNETCORE_ENVIRONMENT=Production`, `ConnectionStrings__GameStudio`

Обмеження безкоштовного плану: після 15 хвилин без запитів сервіс зупиняється, і перший запит після паузи виконується до хвилини. Файлова система контейнера тимчасова, тому після перезапуску БД повертається до версії з репозиторію, а зареєстровані користувачі зникають (ролі й адміністратора застосунок створює заново). Для постійного зберігання даних можна підключити Render PostgreSQL або постійний диск (платні).

2.2. Dockerfile

```
# ---------- збирання ----------
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src

# спершу лише файли проєктів: відновлення пакетів кешується, поки вони не змінюються
COPY GameStudio.Common/GameStudio.Common.csproj GameStudio.Common/
COPY GameStudio.Infrastructure/GameStudio.Infrastructure.csproj GameStudio.Infrastructure/
COPY GameStudio.REST/GameStudio.REST.csproj GameStudio.REST/
RUN dotnet restore GameStudio.REST/GameStudio.REST.csproj

COPY GameStudio.Common/ GameStudio.Common/
COPY GameStudio.Infrastructure/ GameStudio.Infrastructure/
COPY GameStudio.REST/ GameStudio.REST/
RUN dotnet publish GameStudio.REST/GameStudio.REST.csproj -c Release -o /app/publish --no-restore

# ---------- запуск ----------
```

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
WORKDIR /app
COPY --from=build /app/publish .

# база даних SQLite з репозиторію (ЛР3); під час запуску застосунок застосовує міграції та створює ролі й адміністратора
COPY --chown=app:app Database/gamestudio.db Database/gamestudio.db
RUN chown app:app /app/Database

ENV ASPNETCORE_ENVIRONMENT=Production \
    ConnectionStrings__GameStudio="Data Source=/app/Database/gamestudio.db" \
    PORT=10000

# застосунок працює від непривілейованого користувача app
USER app
EXPOSE 10000

# Render передає номер порту в змінній PORT; ASP.NET Core слухає його через ASPNETCORE_HTTP_PORTS
CMD ["sh", "-c", "ASPNETCORE_HTTP_PORTS=${PORT} exec dotnet GameStudio.REST.dll"]
```

Збирання поділено на два етапи: в образі SDK відновлюються пакети й публікується `GameStudio.REST`, а в легший образ ASP.NET Core Runtime копіюються лише результат публікації та файл БД. Застосунок працює від непривілейованого користувача `app`. `.dockerignore` виключає `bin/`, `obj/`, `.git/`, звіти та проєкти, не потрібні REST API (Console, MVC, Nosql, Tests).

2.3. render.yaml (опис інфраструктури)

```
services:
  - type: web
    name: gamestudio-api
    runtime: docker
    plan: free
    region: frankfurt
    branch: lab5
    dockerfilePath: ./Dockerfile
    healthCheckPath: /health
    autoDeploy: true
    buildFilter:
      ignoredPaths:
        - Reports/**
    envVars:
      - key: Seed__AdminEmail
        value: admin@gamestudio.ua
      - key: Seed__AdminPassword
        sync: false
```

`sync: false` означає, що значення секрету не зберігається у файлі: його вводять у панелі Render (Environment), а Render зберігає його в налаштуваннях сервісу й показує замаскованим. Для роботи за зворотним проксі в `Program.cs` додано `UseForwardedHeaders()` (заголовки `X-Forwarded-For` / `X-Forwarded-Proto`).

2.4. Кроки розгортання

- додано `Dockerfile`, `.dockerignore`, `render.yaml`, кінцеву точку `/health`, обробку заголовків проксі; Swagger увімкнено в режимі Production; зміни відправлено в гілку `lab5`;
- у панелі Render створено вебсервіс: New → Web Service → Public Git Repository (адреса репозиторію), гілка `lab5`, мова Docker, регіон Frankfurt, план Free, Health Check Path `/health`;
- у розділі Environment додано змінні `Seed__AdminEmail` і `Seed__AdminPassword` (пароль введено вручну, у коді його немає);
- Render зібрав образ із `Dockerfile` (1 хв 4 с) і запустив контейнер; у журналі видно створення ролей і адміністратора, перевірка `/health` пройшла, статус — Live;
- запити до розгорнутого застосунку виконано на сторінці Swagger UI (розділ 3).

2.5. Скріншоти інфраструктури

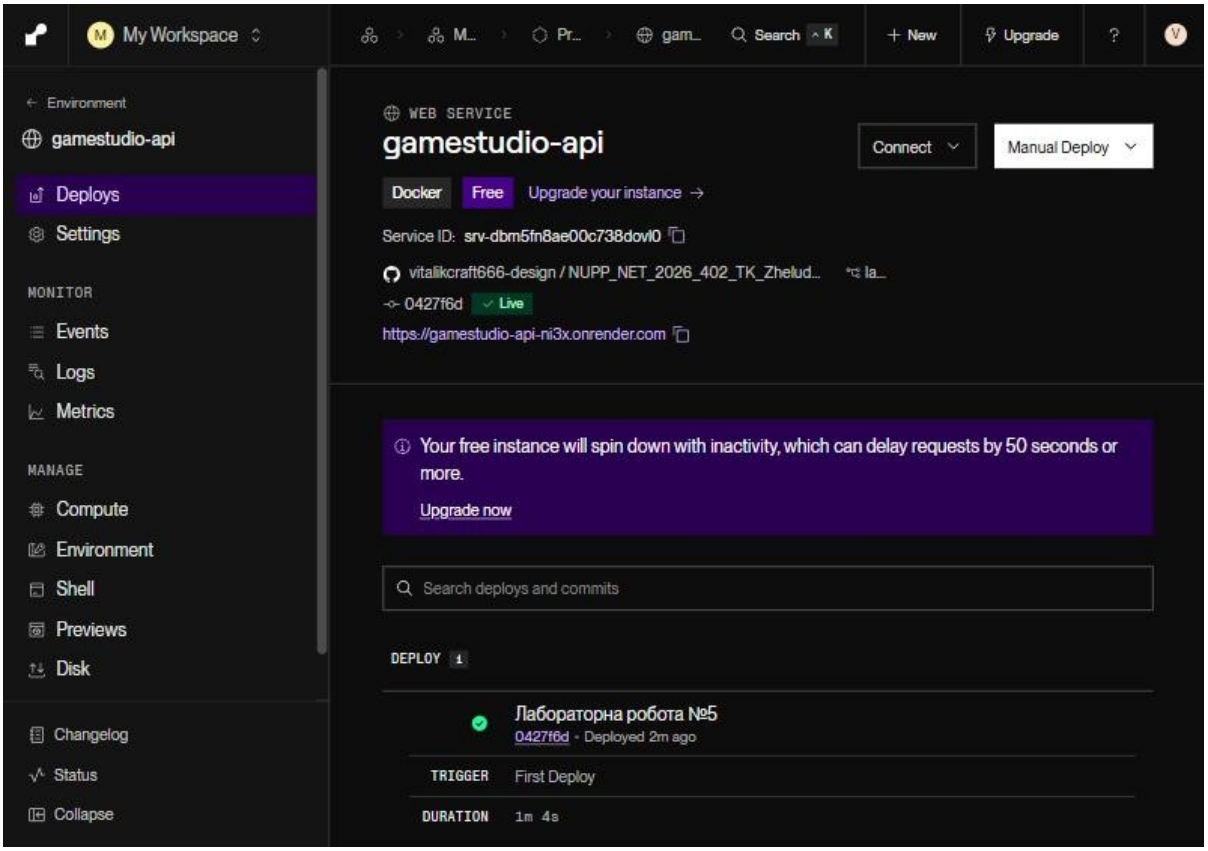


Рисунок 1 — Вебсервіс gamestudio-api на Render: Docker, план Free, комміт 0427f6d, статус Live, адреса застосунку

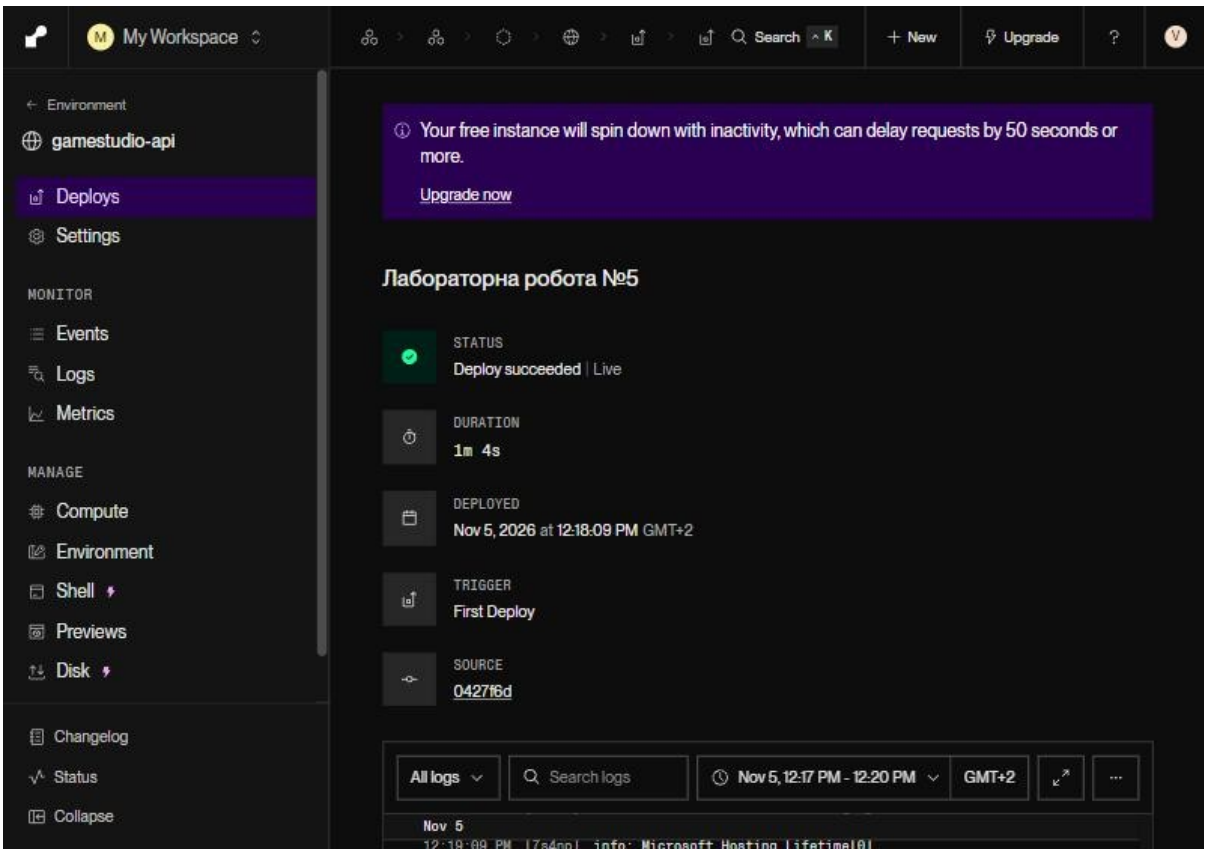


Рисунок 2 — Розгортання «Лабораторна робота №5»: Deploy succeeded, тривалість 1 хв 4 с

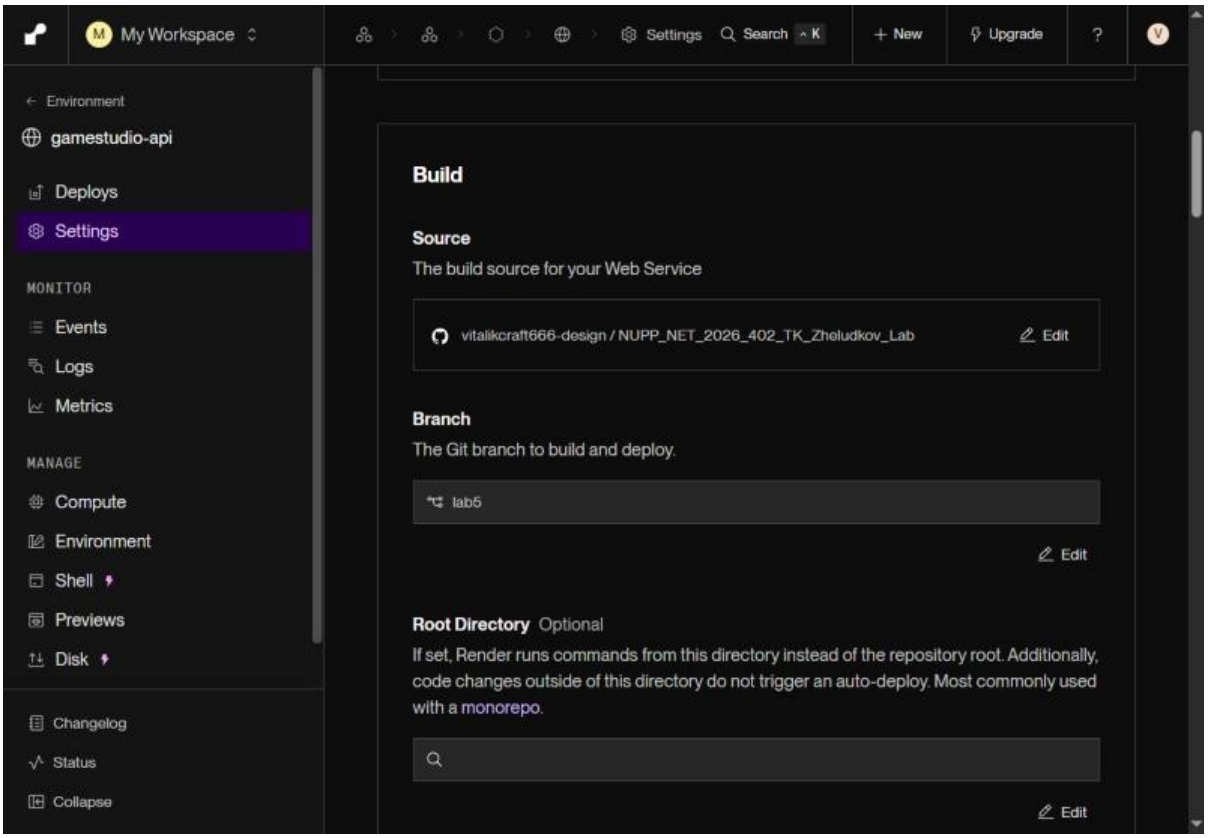


Рисунок 3 — Налаштування збирання: репозиторій GitHub і гілка lab5

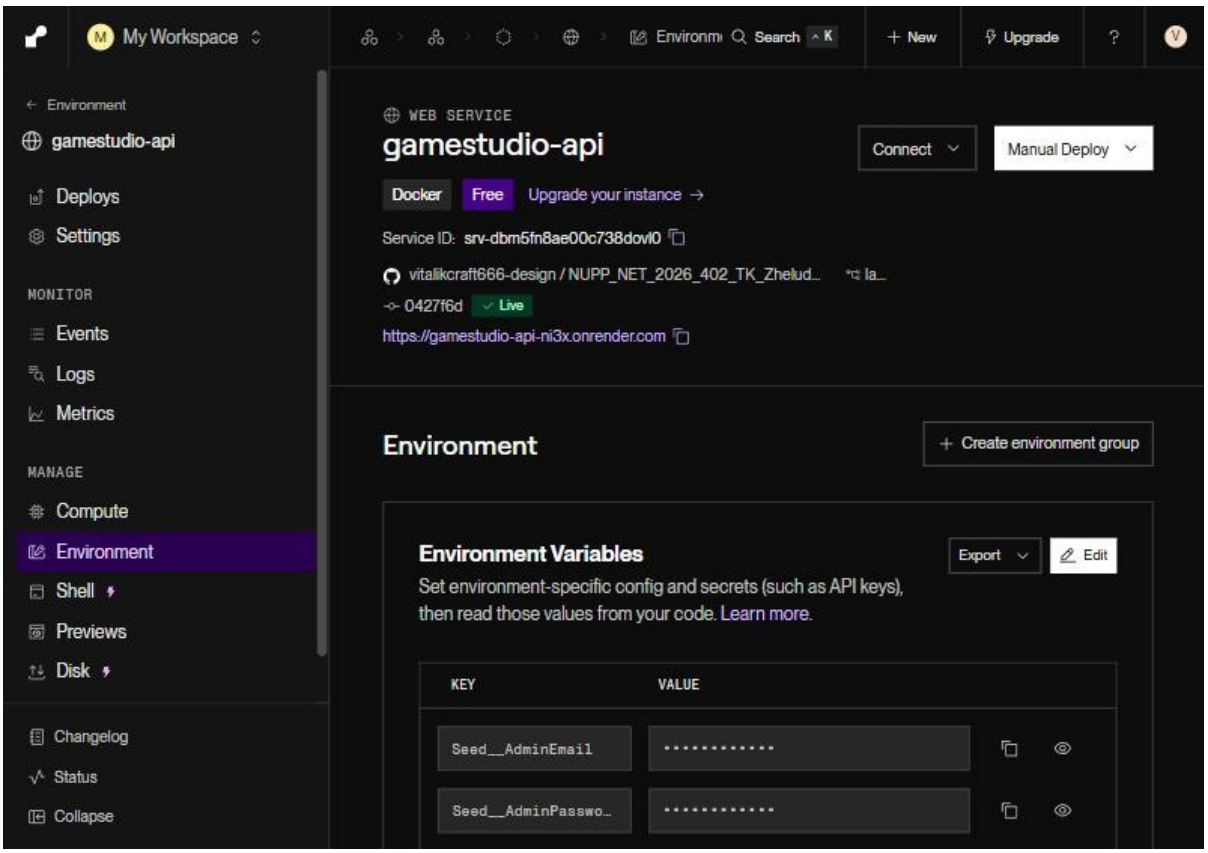


Рисунок 4 — Змінні середовища сервісу: значення (зокрема пароль адміністратора) приховано

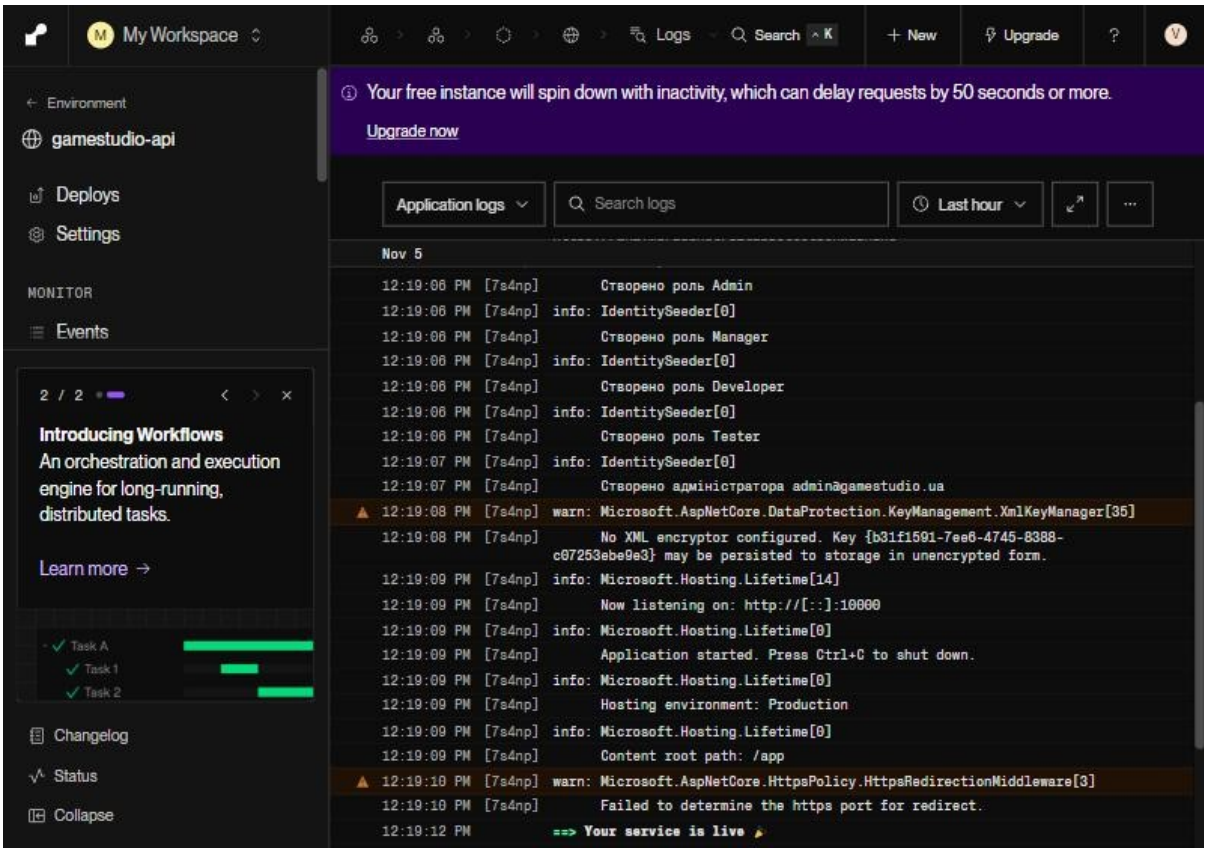


Рисунок 5 — Журнал застосунку на Render: створено ролі Admin, Manager, Developer, Tester і адміністратора; Kestrel слухає порт 10000, середовище Production

3. Результати запитів до розгорнутого застосунку

Усі запити виконано на сторінці Swagger UI розгорнутого застосунку (https://gamestudio-api-ni3x.onrender.com/swagger), тож у полі Request URL кожного скріншоту — адреса на onrender.com. Для запитів від імені користувача токен з відповіді `/api/auth/login` вставлено у вікно **Authorize**, і Swagger додавав його в заголовок `Authorization: Bearer …`. Паролі на скріншотах замасковано, а токени розмито.

№	Клієнт	Запит	Відповідь
1	без входу	GET /api/projects?page=1&amount=3	200 OK
2	без входу	POST /api/projects	401 Unauthorized
3	без входу	GET /api/programmers	401 Unauthorized
4	без входу	GET /api/account/me	401 Unauthorized
5	—	POST /api/auth/register	200 OK
6	—	POST /api/auth/login	200 OK, токени
7	Tester	GET /api/account/me	200 OK
8	Tester	POST /api/bugs	201 Created
9	Tester	POST /api/projects	403 Forbidden
10	Tester	GET /api/programmers	403 Forbidden
11	Admin	GET /api/users	200 OK
12	Admin	PUT /api/users/{id}/roles/Manager	200 OK
13	Admin	PUT /api/users/{id}/roles/Superhero	404 Not Found
14	Admin	POST /api/programmers	201 Created
15	Admin	POST /api/projects	201 Created
16	Admin	DELETE /api/projects/{id}	204 No Content

Усі коди відповідей збіглися з очікуваними.

3.1. Сторінка Swagger розгорнутого застосунку

Захищені операції позначено замком (доки токен не введено у вікні Authorize, замок відкритий), загальнодоступні — без замка. В описі захищених операцій указано ролі, яким дозволено доступ.

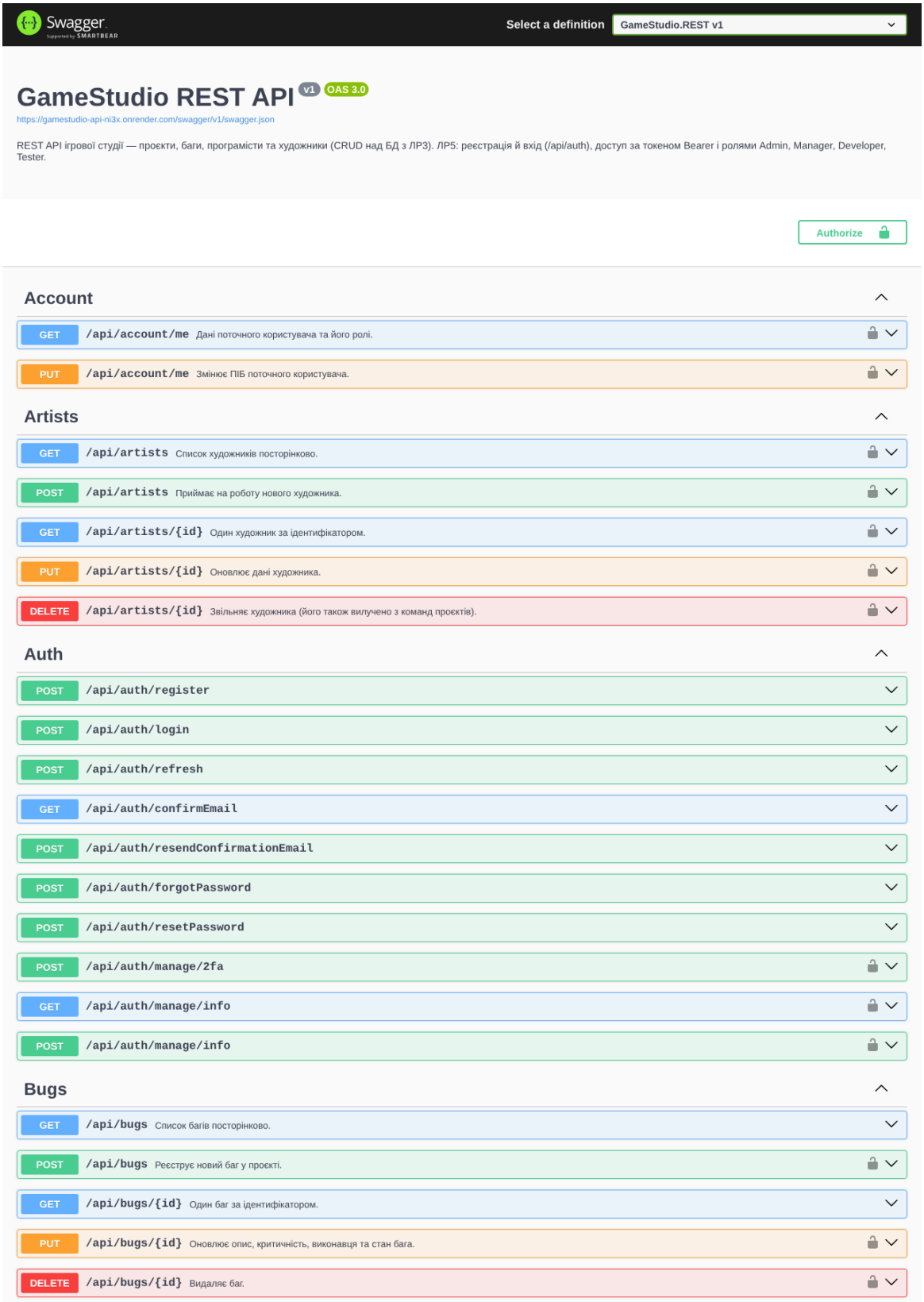


Рисунок 6 — Swagger UI на Render: групи Account, Artists, Auth (вбудовані кінцеві точки Identity), Bugs

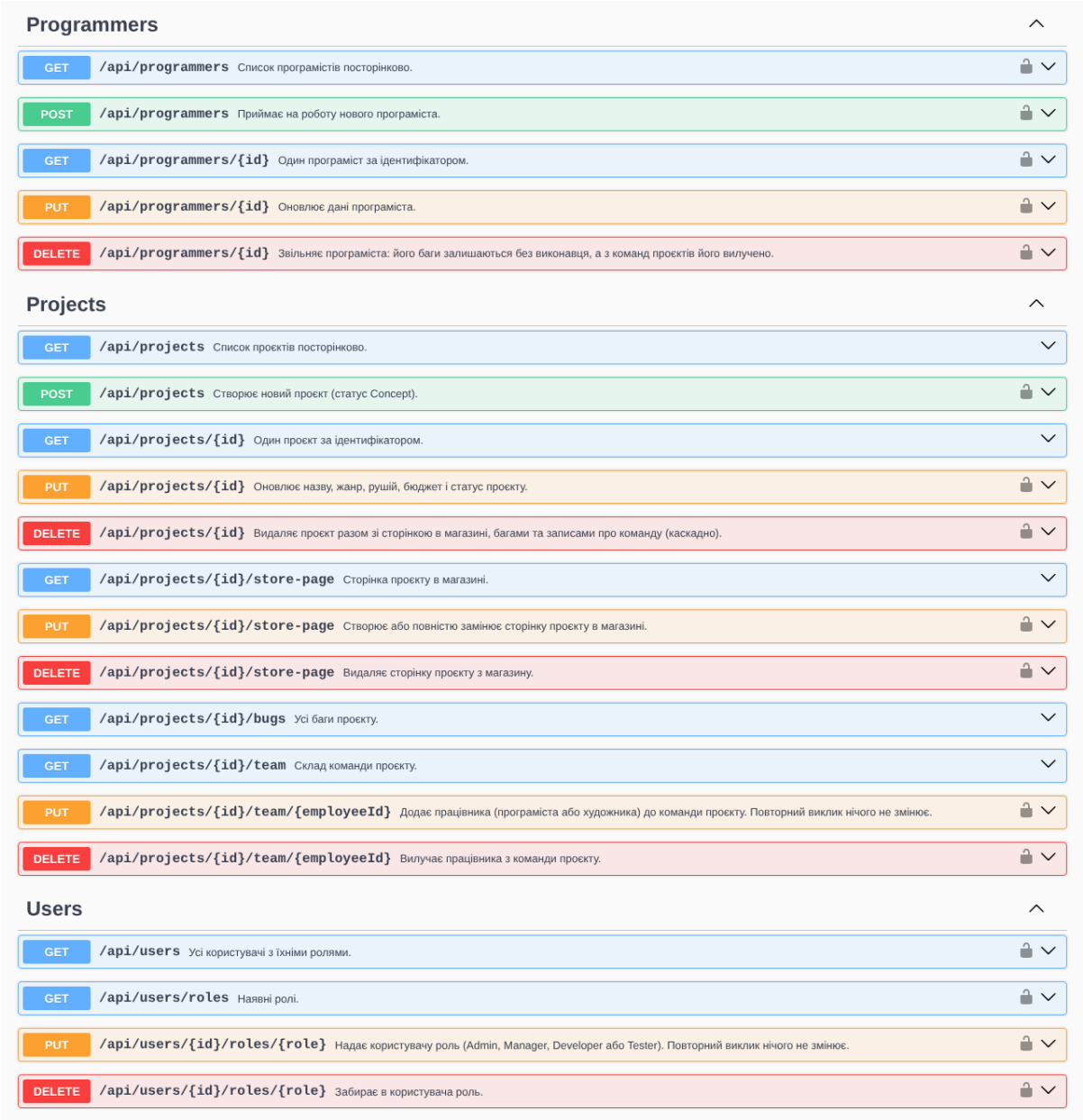


Рисунок 7 — Swagger UI на Render: групи Programmers, Projects, Users

3.2. Клієнт без автентифікації

Запит 1. GET /api/projects → 200 OK

Перегляд проєктів не потребує входу: список повертається і без токена.

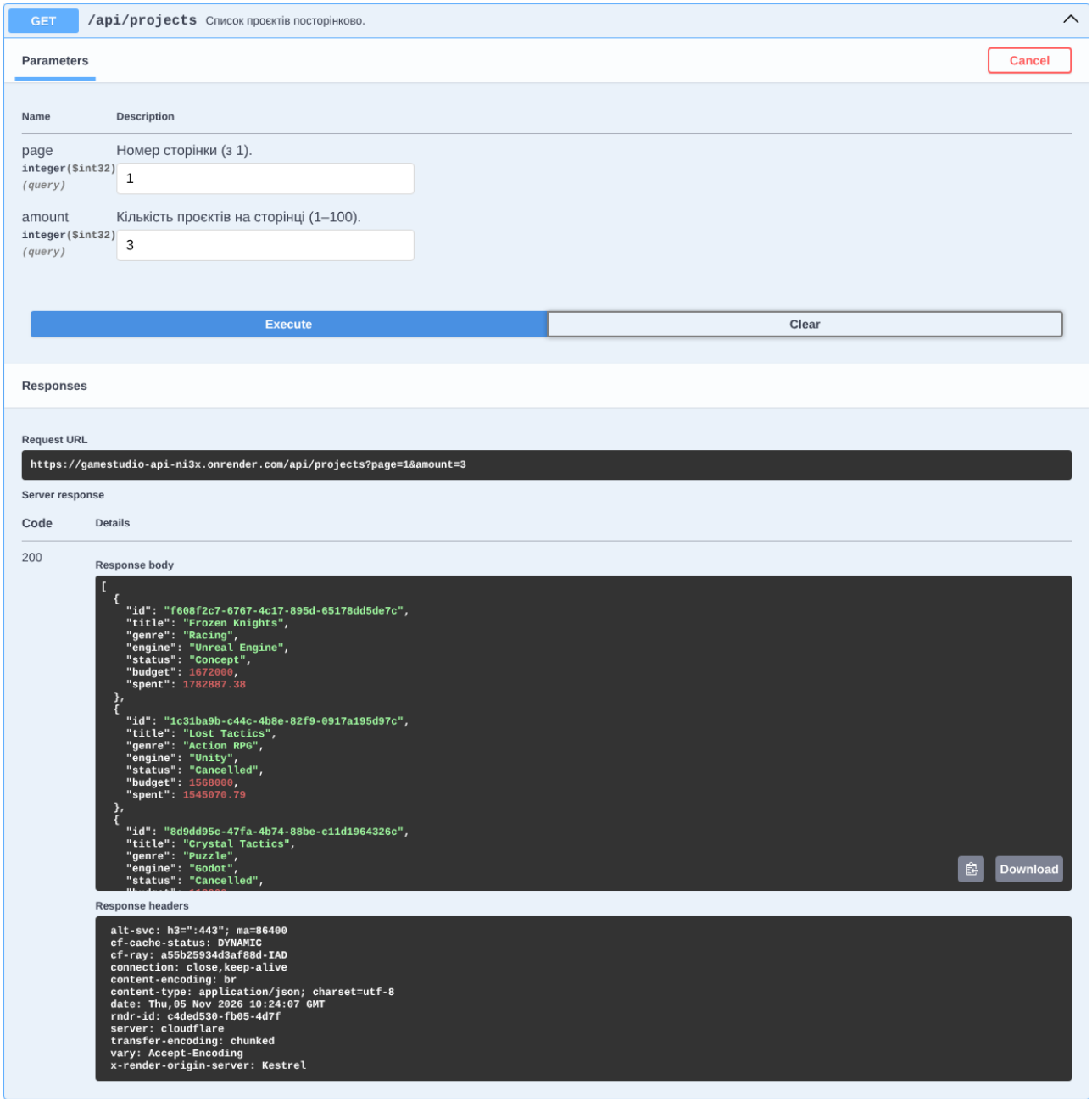


Рисунок 8 — Анонімний клієнт: GET /api/projects — 200 OK

Запит 2. POST /api/projects → 401 Unauthorized

Створення проєкту без токена: 401, заголовок `WWW-Authenticate: Bearer`, тіло ProblemDetails «Потрібна автентифікація».

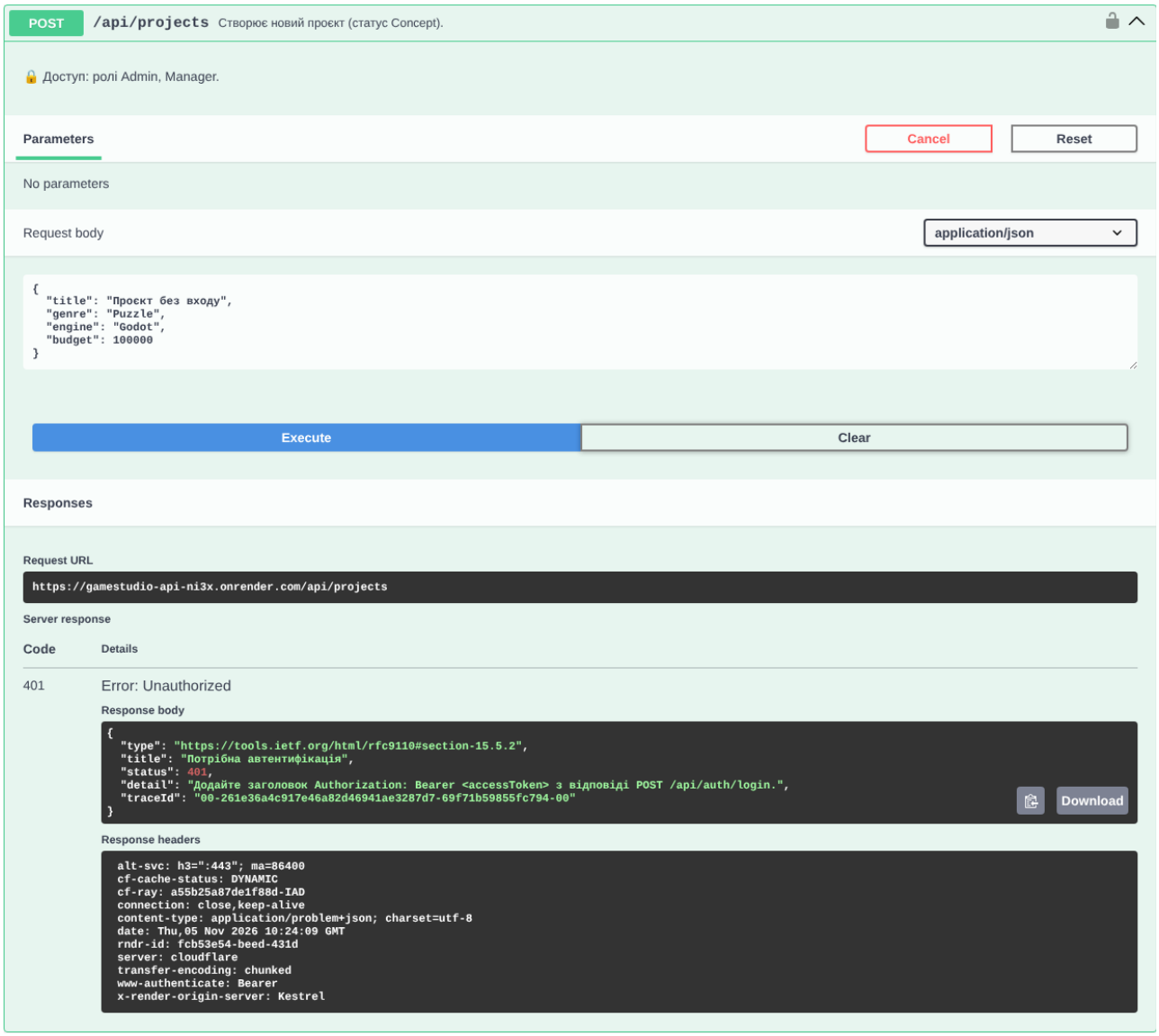


Рисунок 9 — Анонімний клієнт: POST /api/projects — 401 Unauthorized

Запит 3. GET /api/programmers → 401 Unauthorized

Дані працівників (зарплати) доступні лише працівникам студії, тому без токена — 401.

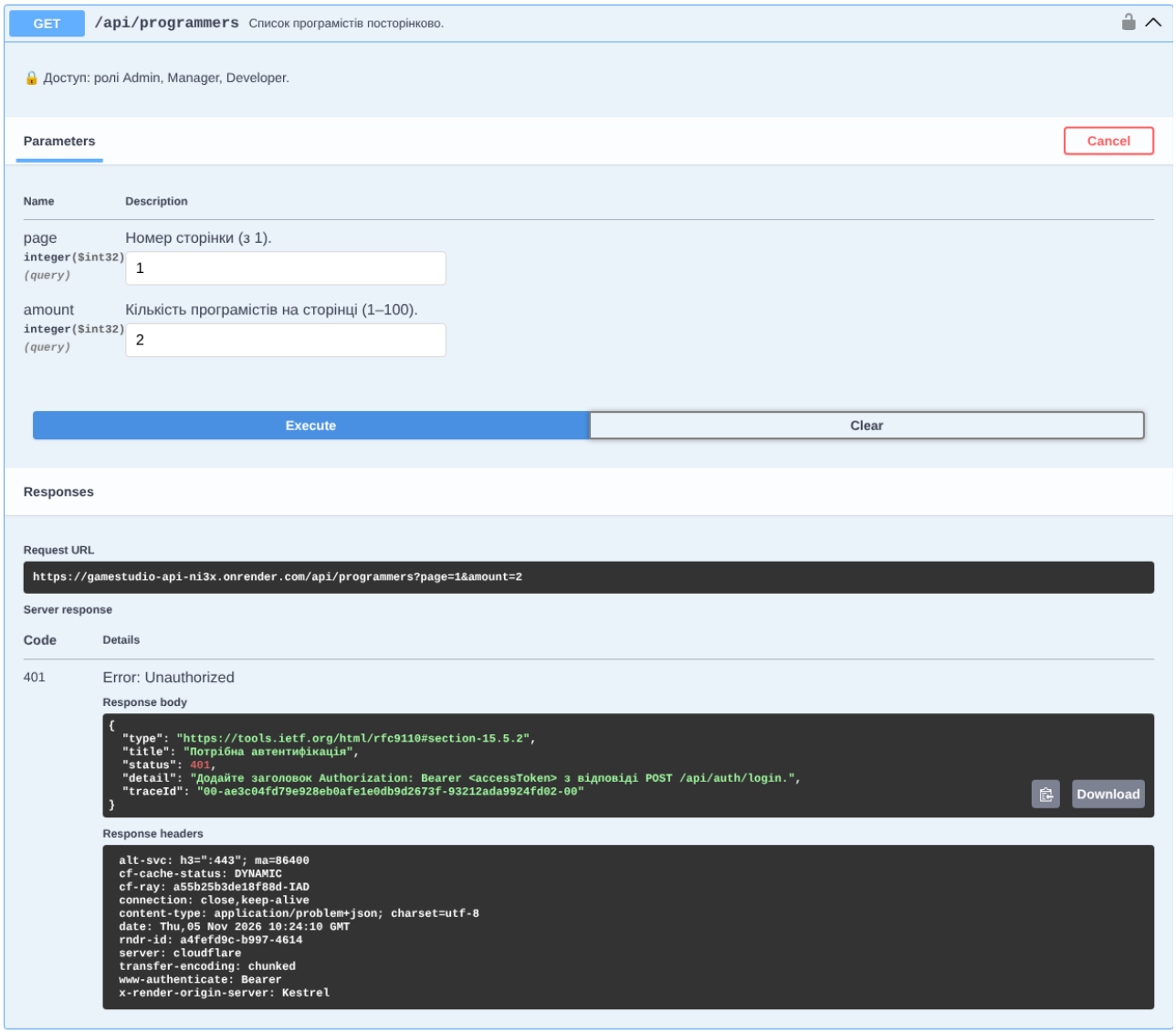


Рисунок 10 — Анонімний клієнт: GET /api/programmers — 401 Unauthorized

Запит 4. GET /api/account/me → 401 Unauthorized

Профіль поточного користувача без токена отримати неможливо.

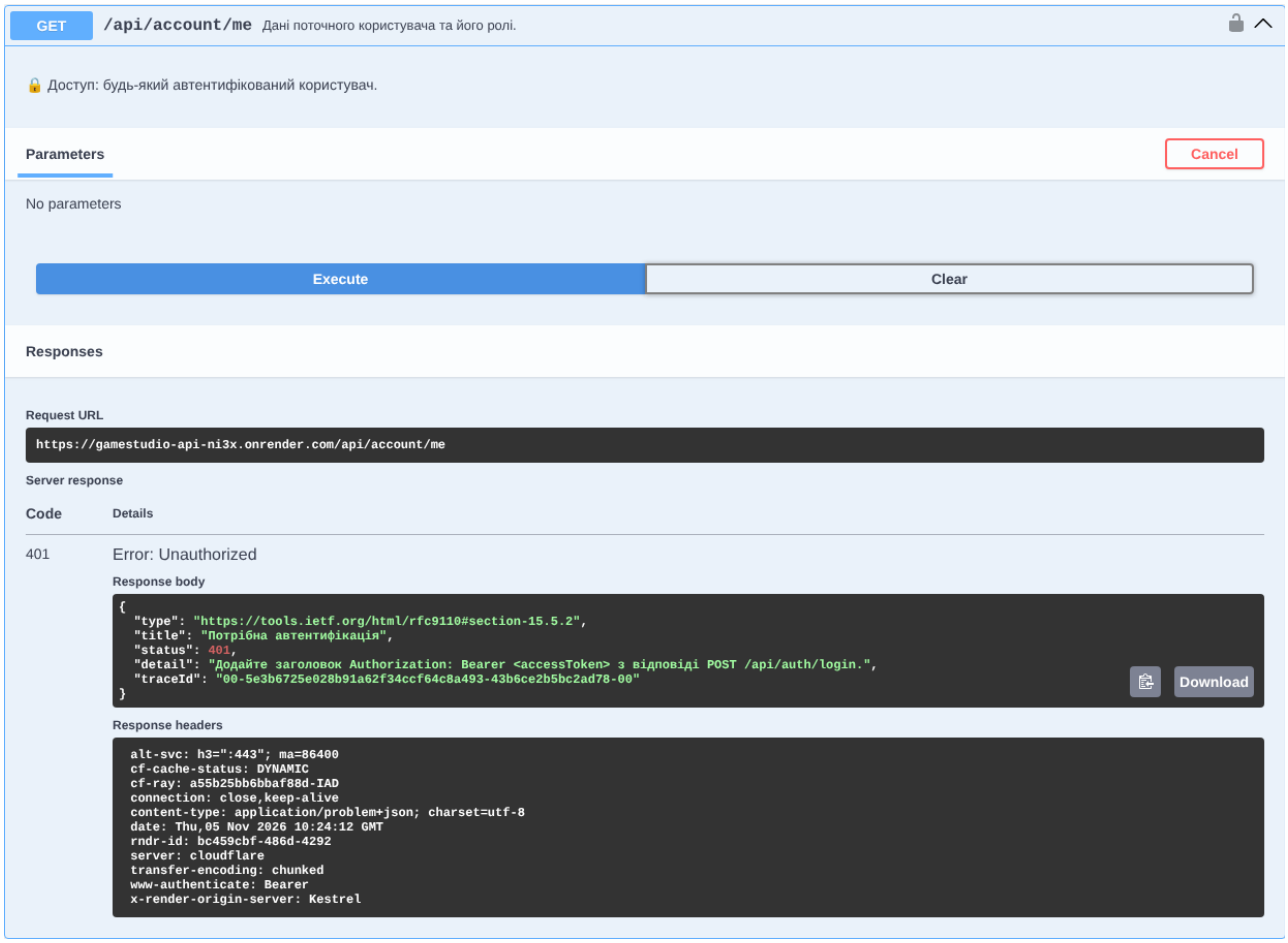


Рисунок 11 — Анонімний клієнт: GET /api/account/me — 401 Unauthorized

3.3. Реєстрація та автентифікація

Запит 5. POST /api/auth/register → 200 OK

Реєстрація користувача `tester@gamestudio.ua` вбудованою кінцевою точкою Identity. Успішна реєстрація повертає 200 без тіла; `StudioUserManager` надає новому користувачу роль Tester.

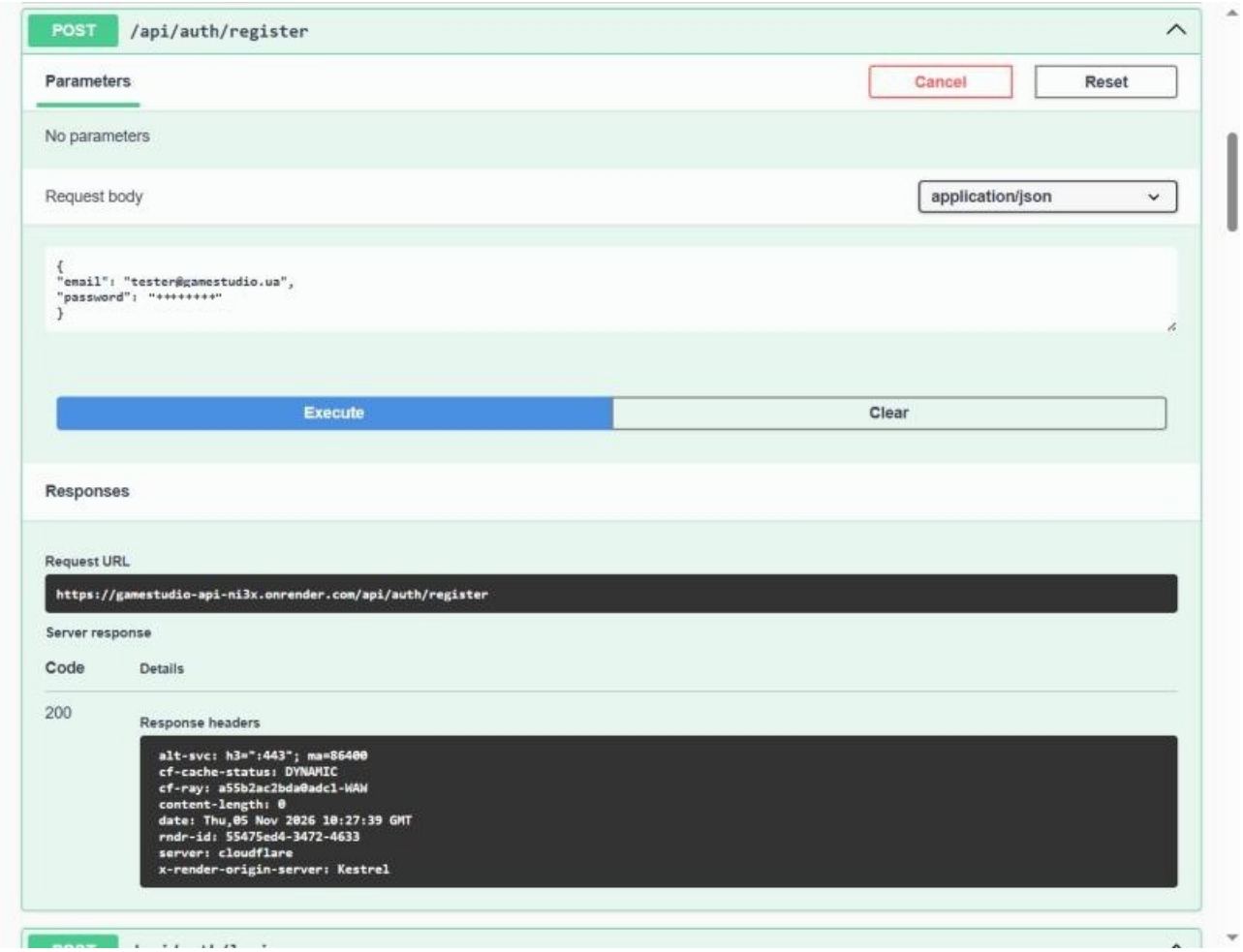


Рисунок 12 — Реєстрація: POST /api/auth/register — 200 OK (пароль замасковано)

Запит 6. POST /api/auth/login → 200 OK

Вхід повертає `tokenType: "Bearer"`, токен доступу `accessToken` (діє 3600 с) і токен оновлення `refreshToken`. Токен доступу вставлено у вікно Authorize.

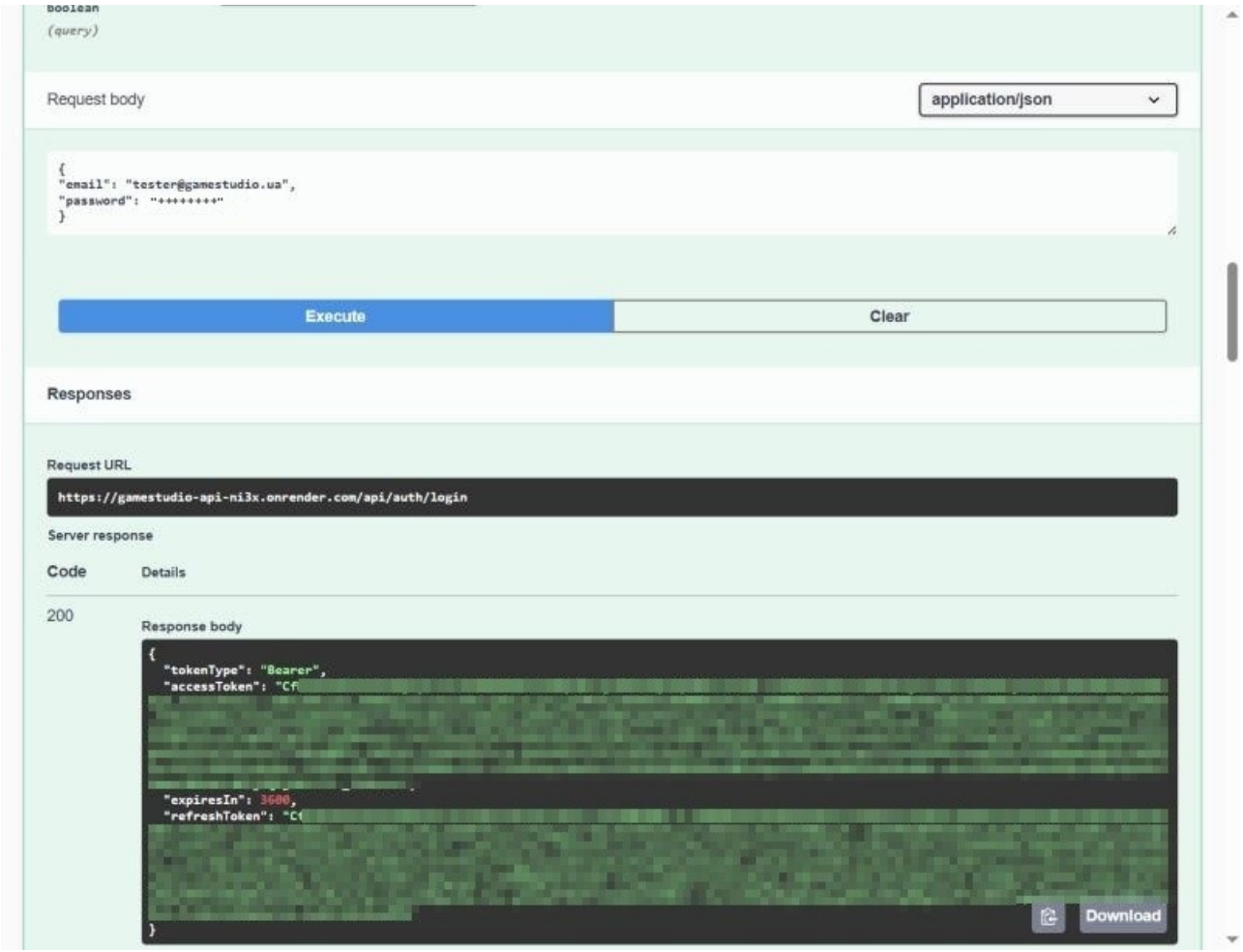


Рисунок 13 — Автентифікація: POST /api/auth/login — 200 OK, токени Bearer (розмито)

3.4. Автентифікований користувач з роллю Tester

Запит 7. GET /api/account/me → 200 OK

Сервер визначив користувача за токеном із заголовка Authorization: роль — Tester.

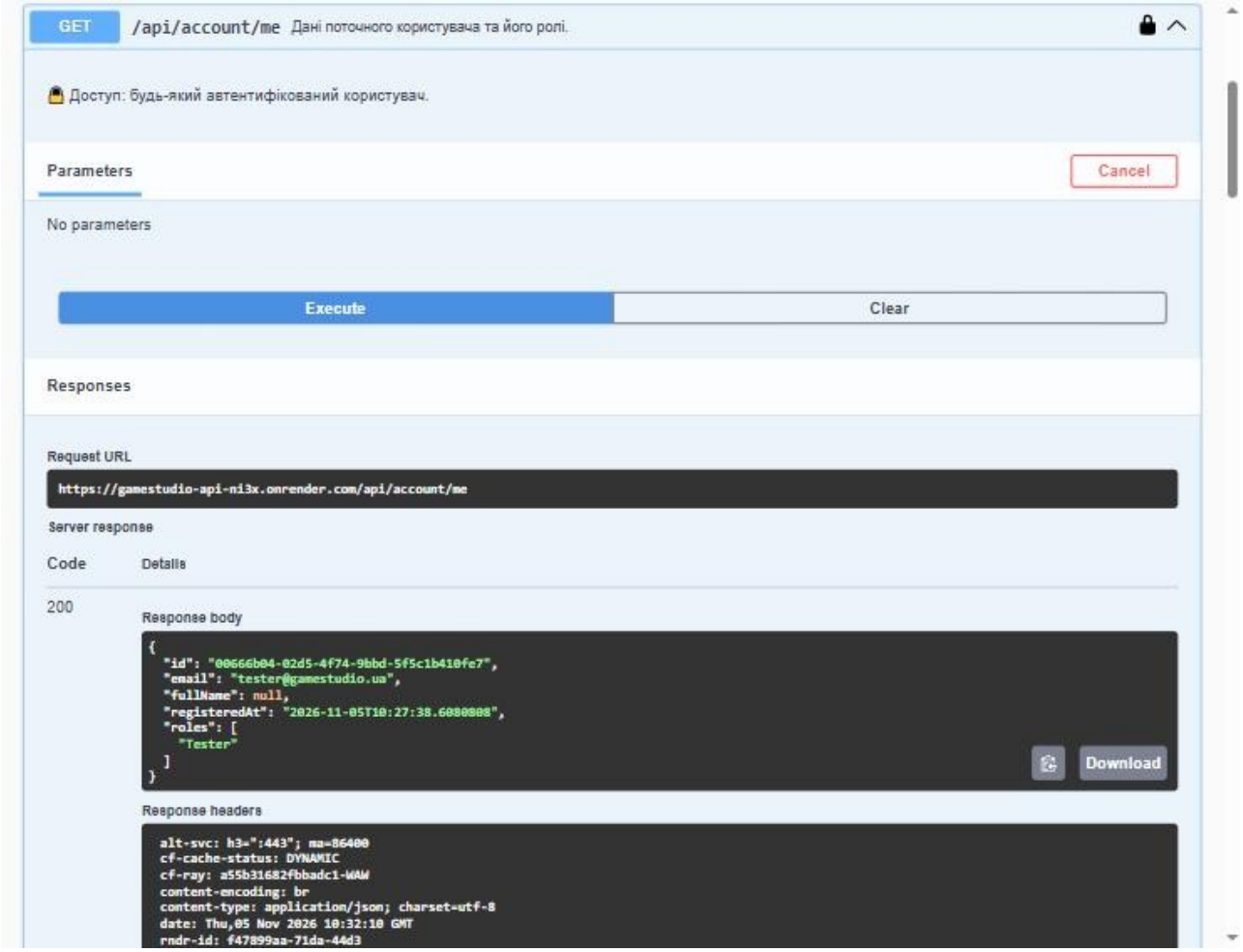


Рисунок 14 — Tester: GET /api/account/me — 200 OK, roles: ["Tester"]

Запит 8. POST /api/bugs → 201 Created

Тестувальник може повідомити про баг у проєкті — дія дозволена будь-якому автентифікованому користувачу.

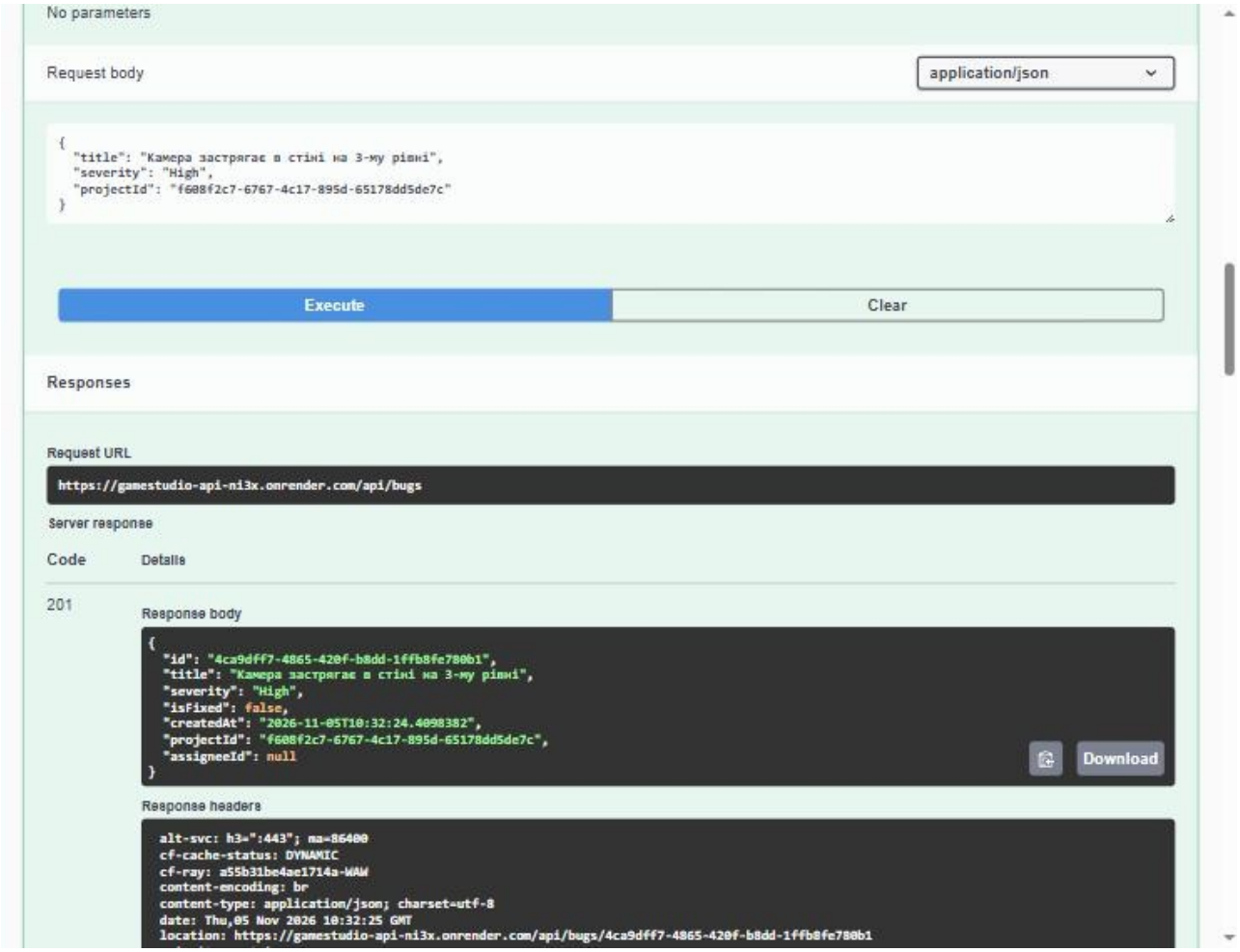


Рисунок 15 — Tester: POST /api/bugs — 201 Created

Недостатній рівень доступу

Запит 9. POST /api/projects → 403 Forbidden

Токен дійсний, але створювати проєкти можуть лише Admin і Manager: 403, тіло ProblemDetails «Доступ заборонено».

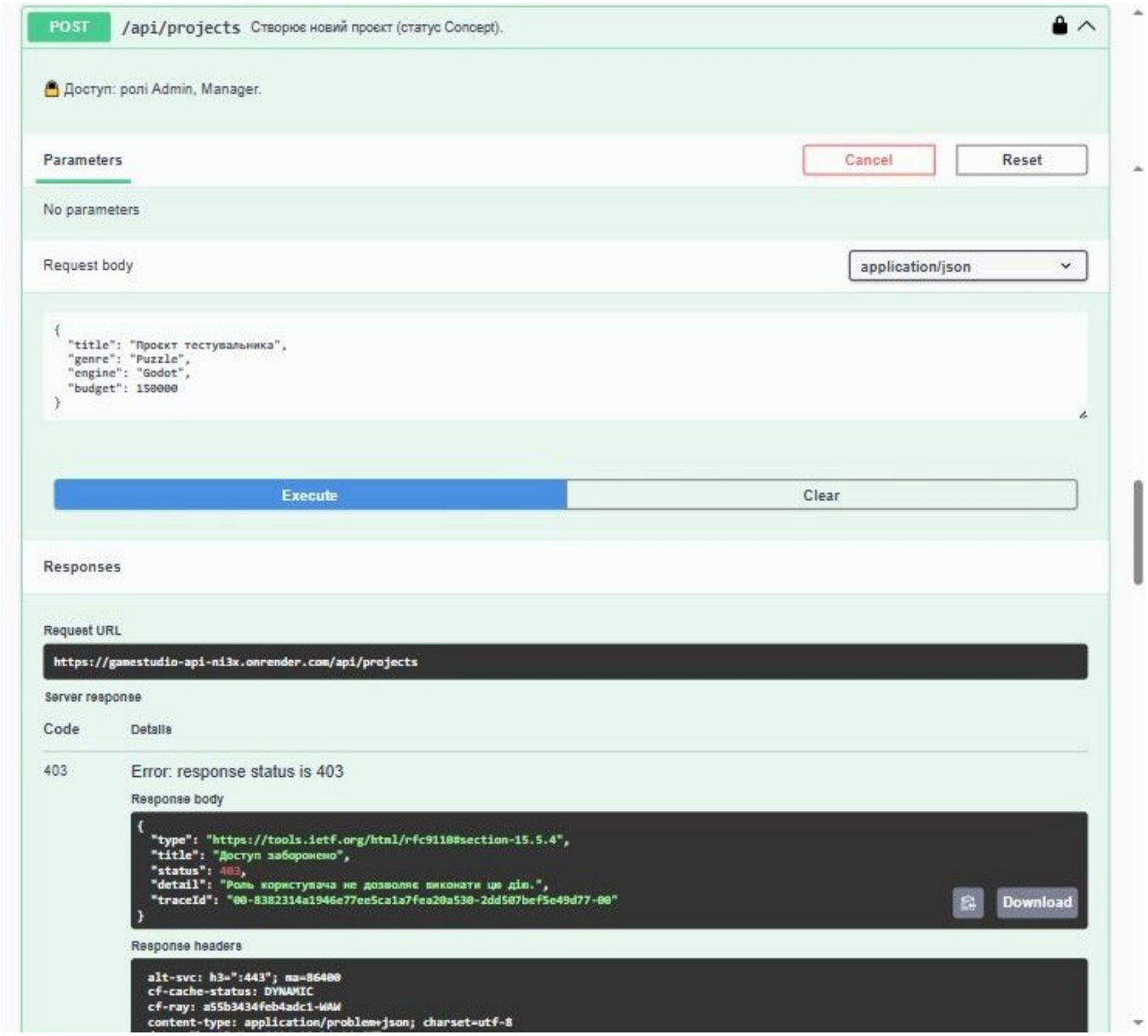


Рисунок 16 — Tester: POST /api/projects — 403 Forbidden

Запит 10. GET /api/programmers → 403 Forbidden

Список програмістів доступний ролям Admin, Manager, Developer; для Tester — 403.

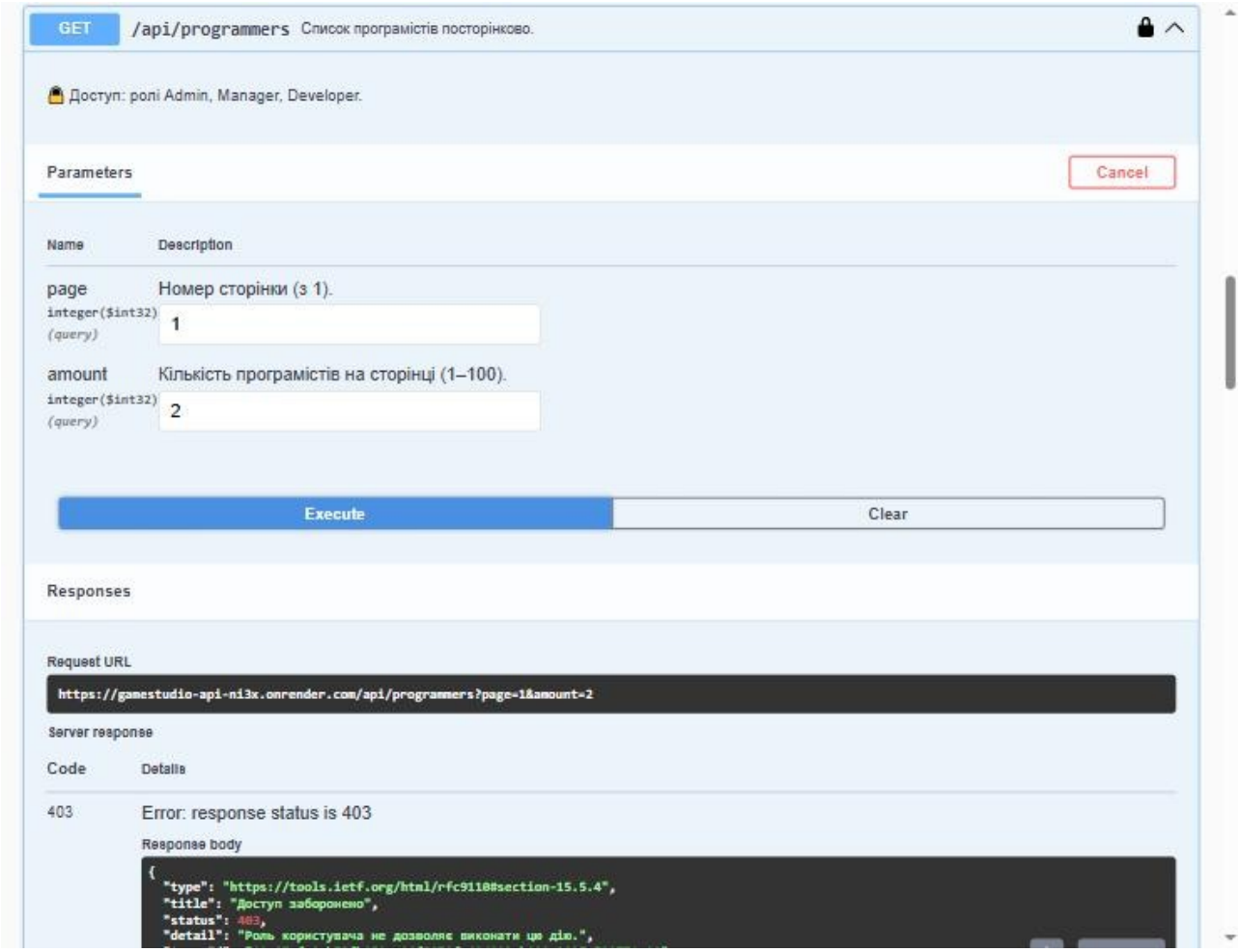


Рисунок 17 — Tester: GET /api/programmers — 403 Forbidden

3.5. Адміністратор (роль Admin)

Адміністратор увійшов через `POST /api/auth/login` з обліковими даними, заданими змінними середовища `Seed__AdminEmail` / `Seed__AdminPassword` на Render (скріншот входу не наведено, бо в тілі запиту пароль). Його токен вставлено у вікно Authorize.

Запит 11. GET /api/users → 200 OK

Список користувачів з ролями доступний лише Admin: адміністратор (Admin, Tester) і зареєстрований тестувальник (Tester).

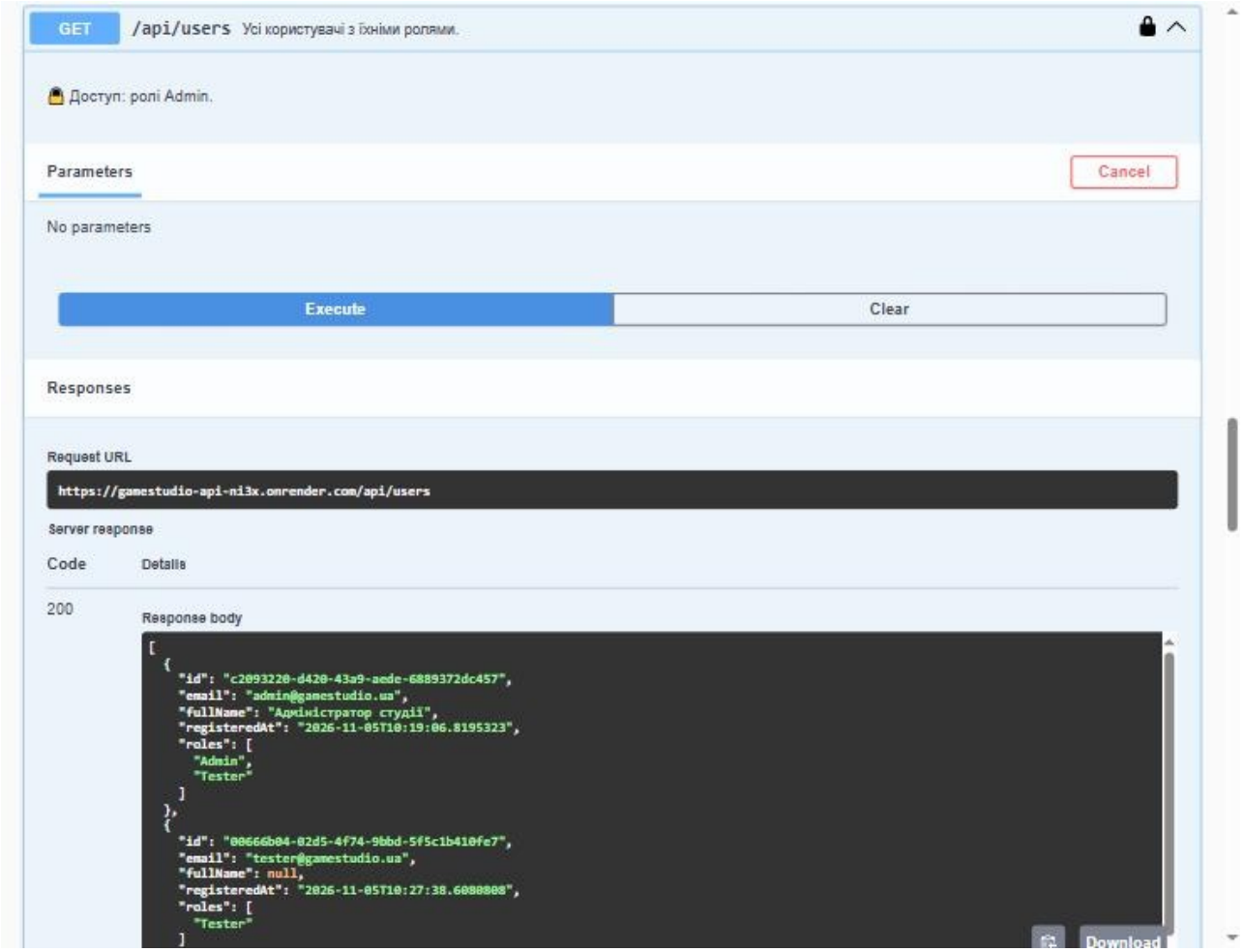


Рисунок 18 — Admin: GET /api/users — 200 OK

Запит 12. PUT /api/users/{id}/roles/Manager → 200 OK

Адміністратор надав тестувальнику роль Manager. Нова роль потрапить у токен користувача після повторного входу або `POST /api/auth/refresh`.

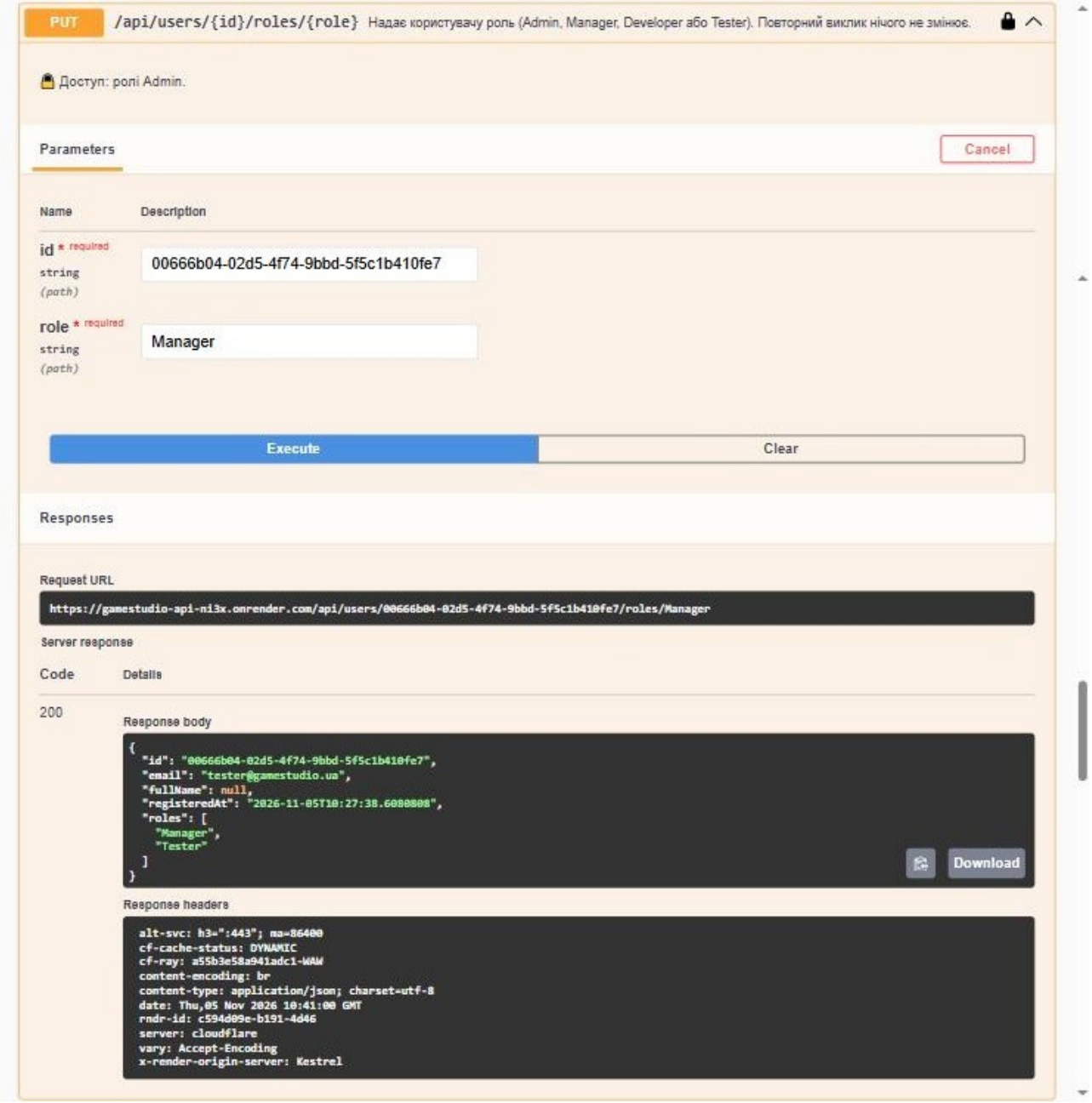


Рисунок 19 — Admin: надання ролі Manager — 200 OK, roles: ["Manager", "Tester"]

Запит 13. PUT /api/users/{id}/roles/Superhero → 404 Not Found

Ролі Superhero не існує: 404 з переліком доступних ролей.

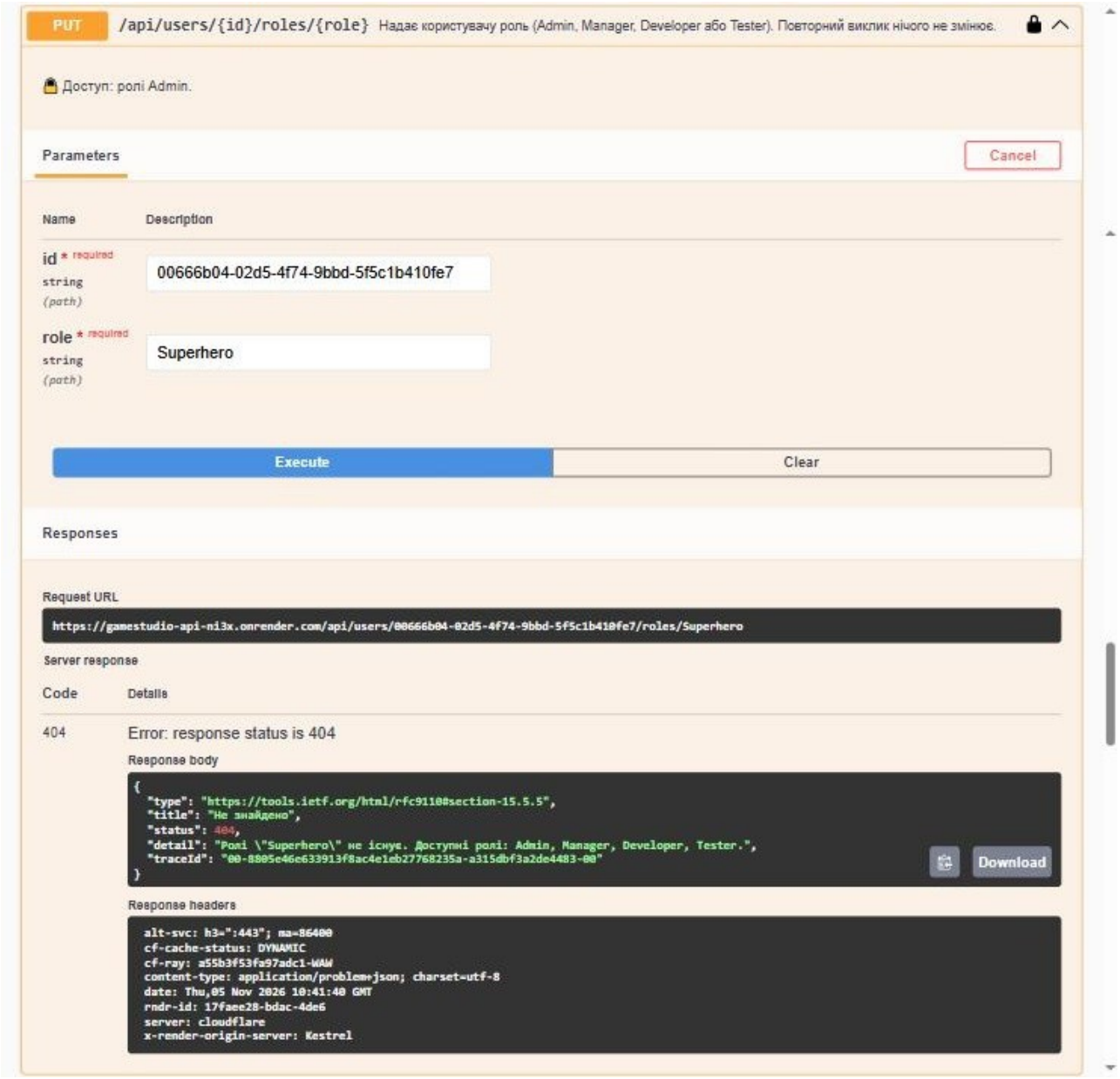


Рисунок 20 — Admin: неіснуюча роль — 404 Not Found

Запит 14. POST /api/programmers → 201 Created

Прийом програміста на роботу дозволено лише Admin.

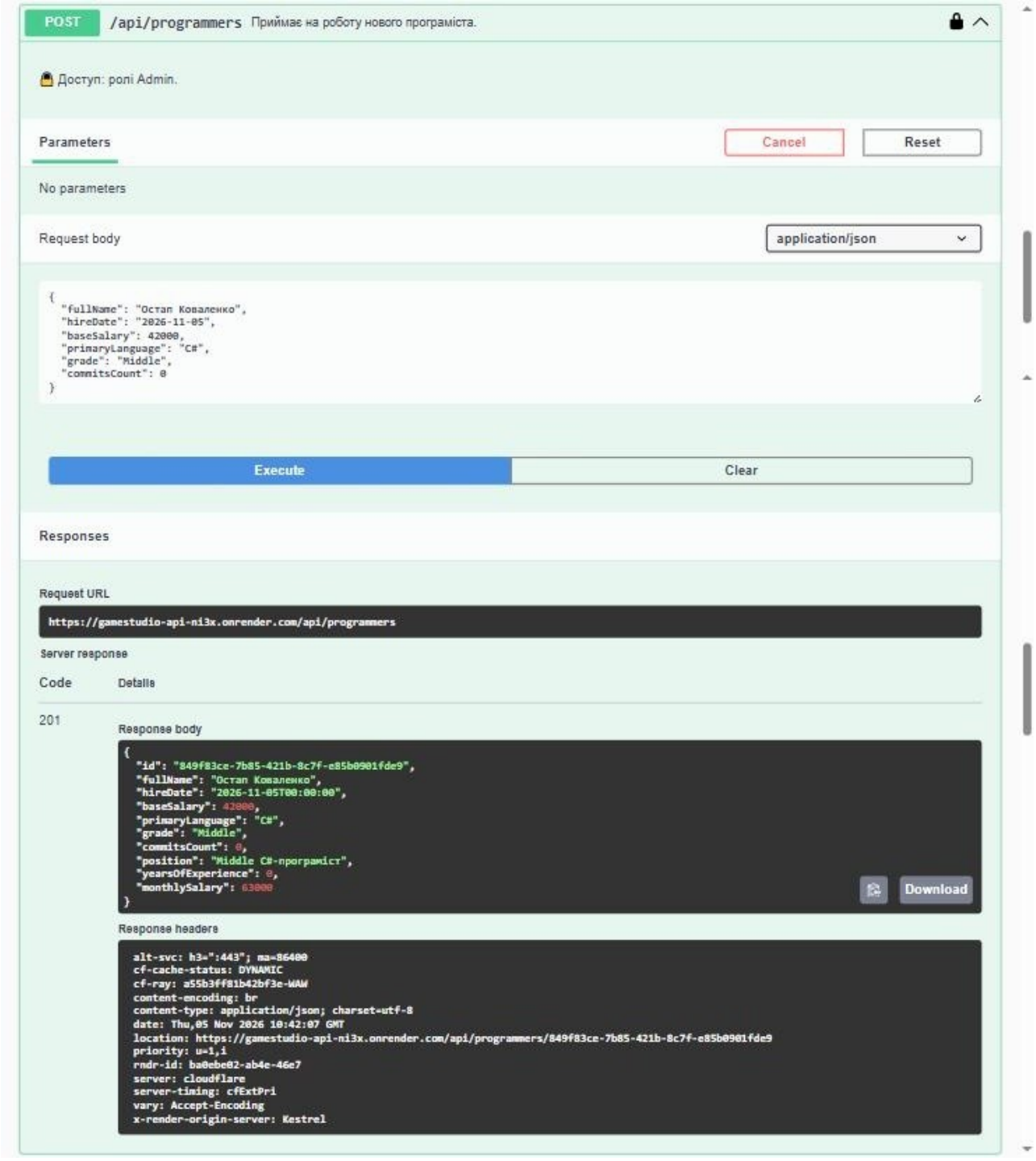


Рисунок 21 — Admin: POST /api/programmers — 201 Created

Запит 15. POST /api/projects → 201 Created

Створення тимчасового проєкту (дозволено Admin і Manager).

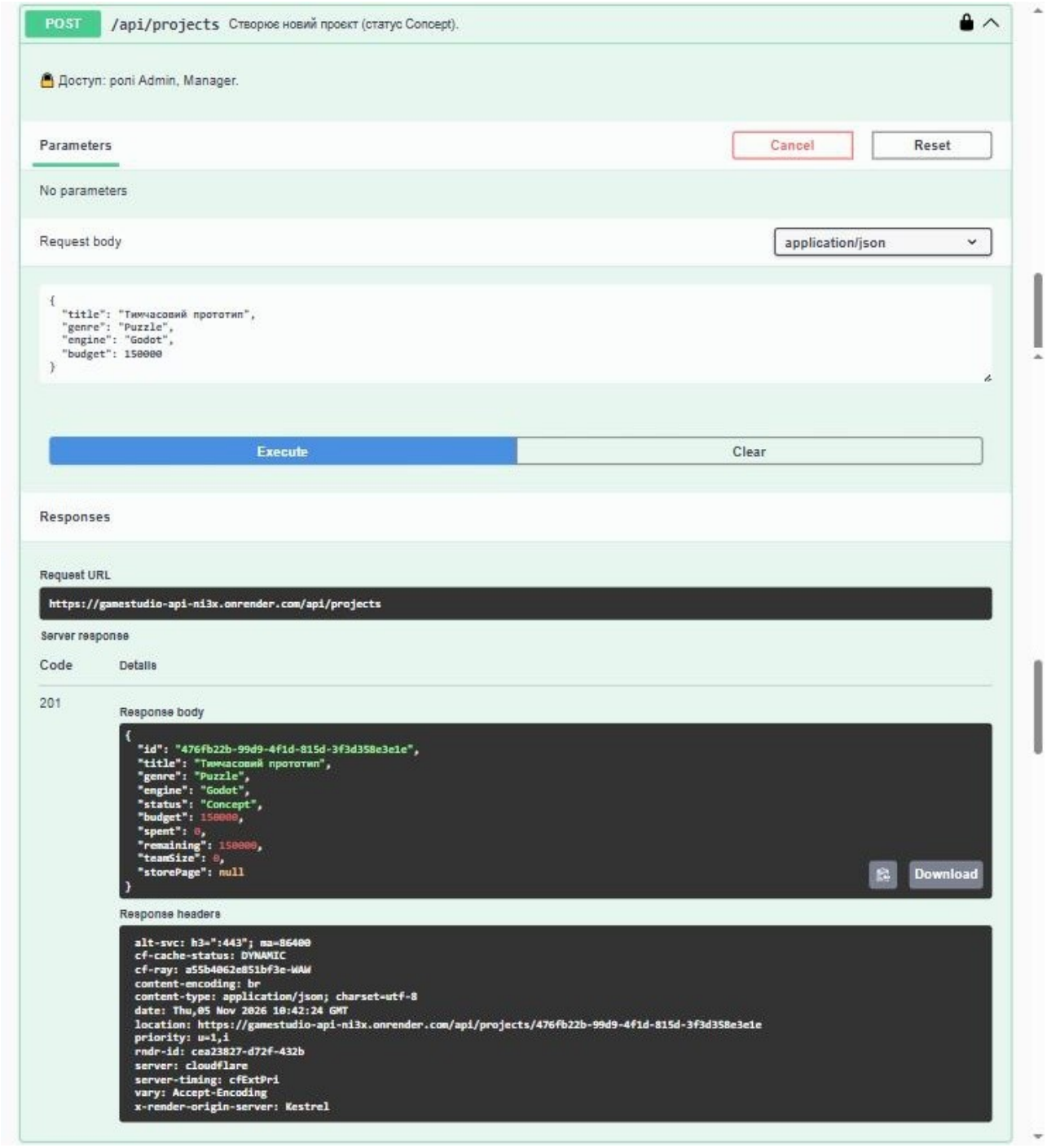


Рисунок 22 — Admin: POST /api/projects — 201 Created

Запит 16. DELETE /api/projects/{id} → 204 No Content

Видалення проєкту дозволено лише Admin; створений проєкт видалено.

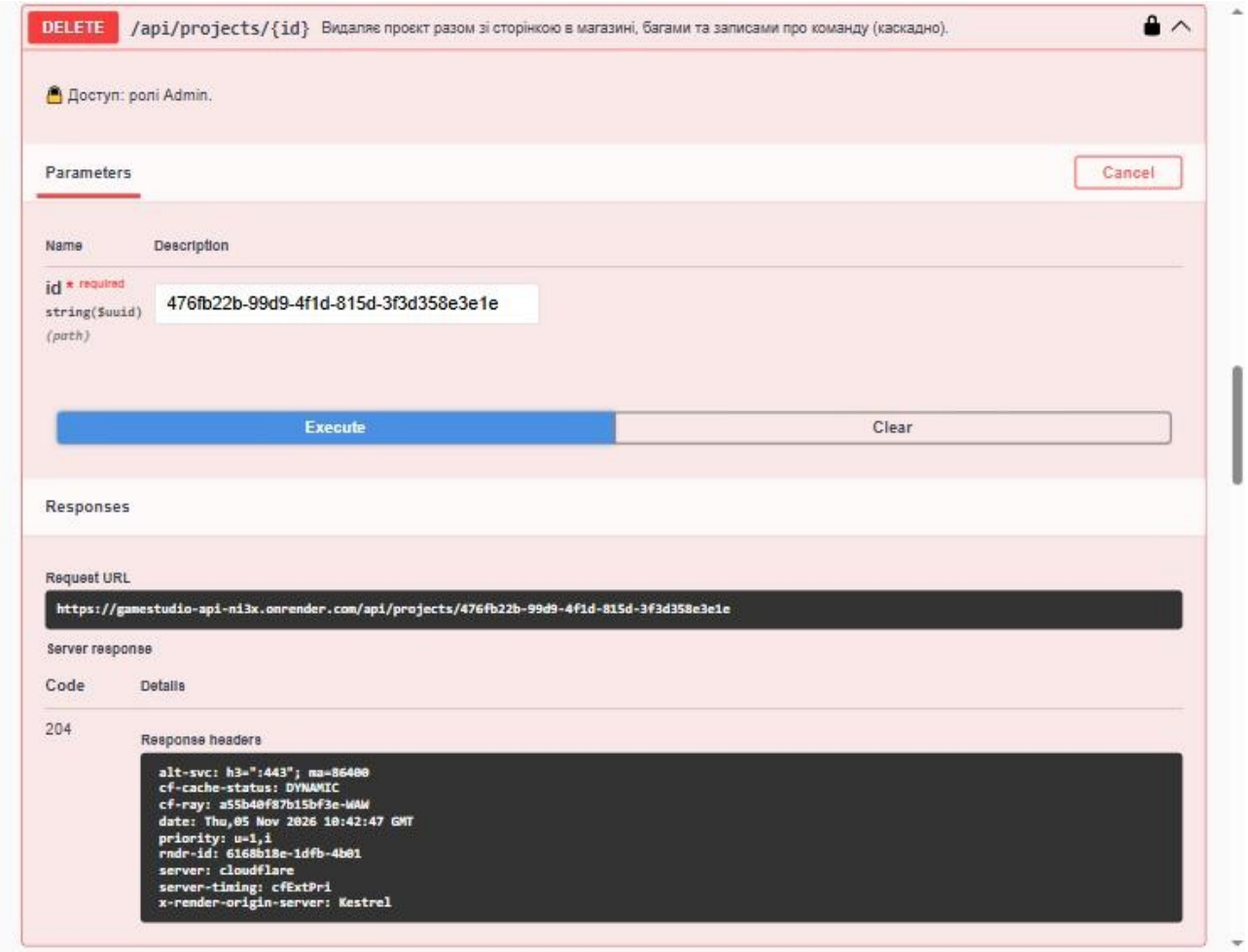


Рисунок 23 — Admin: DELETE /api/projects/{id} — 204 No Content

4. Результати тестів

Команда `dotnet test`: пройдено 72 з 72 тестів. Тести автентифікації та авторизації (`GameStudio.Tests/AuthTests.cs`) запускають застосунок у пам'яті (`WebApplicationFactory<Program>`) з тимчасовою БД і тестовим адміністратором:

```
✔ AuthTests.Register_ThenLogin_ReturnsBearerAndRefreshTokens
✔ AuthTests.Register_RejectsWeakPasswordAndDuplicateEmail
✔ AuthTests.Login_WithWrongPassword_Returns401
✔ AuthTests.NewUser_GetsTesterRole_AndCanUpdateProfile
✔ AuthTests.Anonymous_CanReadPublicData_ButGets401ForProtectedEndpoints
✔ AuthTests.InvalidToken_Returns401
✔ AuthTests.Tester_CanReportBugs_ButGets403ForManagement
✔ AuthTests.Developer_CanUpdateBugs_ButNotProjects
✔ AuthTests.Manager_ManagesProjects_ButOnlyAdminDeletesThemAndHiresEmployees
✔ AuthTests.Admin_ManagesUserRoles
✔ AuthTests.RefreshToken_ReturnsNewAccessTokenWithCurrentRoles

Total tests: 72
     Passed: 72
```

5. Відповіді на контрольні запитання

1. Для чого в проєкт GameStudio.Infrastructure додається пакет Microsoft.AspNetCore.Identity.EntityFrameworkCore, і які можливості він надає?

Пакет зберігає дані ASP.NET Core Identity (користувачів, ролі, клейми, зовнішні входи, токени) у БД через Entity Framework Core. Його додано саме в `GameStudio.Infrastructure`, бо це рівень доступу до даних, де вже є `GameStudioContext` і міграції. Пакет надає:

- базовий контекст `IdentityDbContext<TUser>` (та варіанти з типами ролі й ключа), який описує таблиці `AspNetUsers`, `AspNetRoles`, `AspNetUserRoles` тощо;
- реалізації сховищ на EF Core — `UserStore` і `RoleStore`, які підключає `AddEntityFrameworkStores<GameStudioContext>()`;
- через залежність `Microsoft.Extensions.Identity.Stores` — класи `IdentityUser`, `IdentityRole`, `UserManager<T>`, `RoleManager<T>`, хешування паролів, перевірку паролів і користувачів, блокування, токени підтвердження.

Завдяки цьому реєстрація, вхід, ролі й безпечне зберігання паролів (PBKDF2 із сіллю) працюють без власного коду.

2. Яким чином сутність користувача (User) успадковується від IdentityUser, та які властивості ви додатково додали до цієї сутності?

Клас `StudioUser : IdentityUser` (папка `GameStudio.Infrastructure/Identity`). `IdentityUser` — це `IdentityUser<string>`, тож первинний ключ — рядок з Guid. Успадковано: `Id`, `UserName`, `NormalizedUserName`, `Email`, `NormalizedEmail`, `EmailConfirmed`, `PasswordHash`, `SecurityStamp`, `ConcurrencyStamp`, `PhoneNumber`, `PhoneNumberConfirmed`, `TwoFactorEnabled`, `LockoutEnd`, `LockoutEnabled`, `AccessFailedCount`.

Додано: **`FullName`** — ПІБ (необов'язкове, до 100 символів через `StudioUserConfiguration`, змінюється через `PUT /api/account/me`) і **`RegisteredAt`** — дата й час реєстрації в UTC, яку заповнює `StudioUserManager.CreateAsync`. Обидва поля стали стовпцями таблиці `AspNetUsers` і повертаються в моделі API `AccountModel` разом із ролями.

3. Чому DbContext потрібно наслідувати від IdentityDbContext<TUser>, і які зміни це вносить у конфігурацію бази даних?

Сховища Identity (`UserStore`, `RoleStore`) працюють з контекстом, у якому є набори `Users`, `Roles`, `UserRoles`, `UserClaims`, `UserLogins`, `UserTokens`, `RoleClaims`. Ці набори та їхню конфігурацію містить `IdentityDbContext<StudioUser>` (скорочення для `IdentityDbContext<StudioUser, IdentityRole, string>`). Успадкувавши його, `GameStudioContext` зберігає користувачів у тій самій БД, що й дані ЛР3, а `AddEntityFrameworkStores<GameStudioContext>()` може використати цей контекст.

Зміни в БД: `IdentityDbContext.OnModelCreating` описує сім таблиць `AspNet*` з ключами, унікальними індексами (`UserNameIndex`, `EmailIndex`, `RoleNameIndex`), обмеженнями довжини та зв'язками. Тому в нашому `OnModelCreating` спершу викликається

`base.OnModelCreating(modelBuilder)`, а потім застосовуються конфігурації ЛР3 і `StudioUserConfiguration`. Нову схему оформлено міграцією `AddIdentity`: вона лише створює таблиці Identity, а таблиці й дані ЛР3 не змінюються.

4. Як налаштовуються служби Identity у проєкті GameStudio.REST, зокрема менеджери користувачів (UserManager) та ролей (RoleManager)?

Усе налаштовано в `Program.cs` (код у розділі 1.3):

- `AddIdentityApiEndpoints<StudioUser>(options)` реєструє `UserManager<StudioUser>`, `SignInManager<StudioUser>`, хешувальник паролів, валідатори, постачальників токенів, схему автентифікації Bearer і сервіси для `MapIdentityApi`. У параметрах — унікальний email, мінімальна довжина пароля 8 і блокування після 5 невдалих спроб;
- `AddRoles<IdentityRole>()` реєструє `RoleManager<IdentityRole>` і фабрику `UserClaimsPrincipalFactory<StudioUser, IdentityRole>`, яка додає ролі користувача в його клейми, а отже, і в токен;
- `AddUserManager<StudioUserManager>()` замінює стандартний `UserManager` нашим нащадком, який під час створення користувача заповнює `RegisteredAt` і додає роль Tester;
- `AddEntityFrameworkStores<GameStudioContext>()` реєструє сховища користувачів і ролей на EF Core;
- `Configure<BearerTokenOptions>` задає час життя токена доступу (1 година).

Менеджери отримуються через впровадження залежностей: `IdentitySeeder` під час запуску використовує `RoleManager.RoleExistsAsync` / `CreateAsync` і `UserManager.CreateAsync` / `AddToRoleAsync`. `AccountController` використовує `UserManager.GetUserAsync(User)`, `GetRolesAsync`, `UpdateAsync`. `UsersController` — `UserManager.Users`, `FindByIdAsync`, `IsInRoleAsync`, `AddToRoleAsync`, `RemoveFromRoleAsync` і `RoleManager.Roles`, `RoleExistsAsync`.

5. Які кроки потрібні для створення та підписування JWT або Bearer-токенів в ASP.NET Core?

У цій роботі — токени Bearer ASP.NET Core Identity (вимога: «Bearer або JWT»). Кроки:

- зареєструвати `AddIdentityApiEndpoints<StudioUser>()` — він додає схему `BearerToken` (`IdentityConstants.BearerScheme`);
- відкрити `MapIdentityApi<StudioUser>()`. `POST /login` перевіряє пароль через `SignInManager.PasswordSignInAsync`, створює `ClaimsPrincipal` (Id, ім'я, email, ролі, `SecurityStamp`) і викликає `SignIn` для схеми Bearer;
- обробник Bearer серіалізує дані автентифікації й захищає їх через **ASP.NET Core Data Protection**: шифрування AES-256-CBC і підпис HMAC-SHA256 ключем сервера. Отриманий рядок — `accessToken`; окремо створюється `refreshToken`;
- налаштувати термін дії через `BearerTokenOptions` (`BearerTokenExpiration`, `RefreshTokenExpiration`).

Для JWT потрібно:

- встановити пакет `Microsoft.AspNetCore.Authentication.JwtBearer`;
- зберегти секретний ключ підпису (не менше 256 біт) у конфігурації або сховищі секретів, а не в коді;
- зареєструвати схему: `AddAuthentication().AddJwtBearer(o => o.TokenValidationParameters = new() { ValidIssuer = …, ValidAudience = …, IssuerSigningKey = new SymmetricSecurityKey(key), ValidateLifetime = true });`
- у власній кінцевій точці входу після перевірки пароля сформувати клейми (`sub`, `email`, `role`), створити `SigningCredentials(key, SecurityAlgorithms.HmacSha256)` і токен з `iss`, `aud`, `exp` (`JwtSecurityTokenHandler` або `JsonWebTokenHandler`).

Відмінність: вміст JWT може прочитати будь-хто (він лише підписаний і закодований Base64Url), а токен Identity непрозорий — і підписаний, і зашифрований.

6. Яким чином налаштовується Middleware для автентифікації та авторизації на основі JWT у вашому REST API?

Схеми реєструються в `builder.Services` (`AddIdentityApiEndpoints` додає схему Bearer, `AddAuthorization` — політики), а в конвеєр додаються два проміжні компоненти в такому порядку:

```
app.UseForwardedHeaders();
app.UseExceptionHandler();
app.UseStatusCodePages();
app.UseSwagger();
app.UseSwaggerUI(options => options.DocumentTitle = "GameStudio REST API");
app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();
app.MapGroup("/api/auth")
    .WithTags("Auth")
    .MapIdentityApi<StudioUser>();
app.MapControllers();
```

- `UseAuthentication()` для кожного запиту викликає обробник схеми за замовчуванням. Він читає заголовок `Authorization`, перевіряє токен і записує користувача в `HttpContext.User`. Без токена запит іде далі як анонімний;
- `UseAuthorization()` читає метадані кінцевої точки (`[Authorize]`, `[Authorize(Roles = …)]`, `[AllowAnonymous]`) і перевіряє політику. Якщо користувача немає — Challenge (401), якщо роль не підходить — Forbid (403);
- порядок важливий: спершу визначаємо, хто робить запит, потім — чи можна йому; обидва компоненти стоять після маршрутизації та перед `MapControllers` і `MapIdentityApi`;
- `UseForwardedHeaders()` стоїть першим, щоб за проксі Render правильно визначалися схема HTTPS та IP клієнта; `UseStatusCodePages()` додає тіло до відповідей 401/403.

Для JWT конвеєр був би таким самим, змінилася б лише реєстрація схеми (`AddJwtBearer`).

7. Що містить HTTP-заголовок Authorization, і як його обробляє сервер при отриманні запиту?

Заголовок має формат `Authorization: <схема> <облікові дані>`. У нашому API це `Authorization: Bearer <accessToken>`, де токен — рядок з відповіді `POST /api/auth/login`. Bearer означає, що доступ отримує той, хто пред'явив токен, тому його передають лише через HTTPS.

Обробка на сервері:

- `AuthenticationMiddleware` викликає обробник Bearer, який бере значення після `Bearer ` ;
- токен розшифровується й перевіряється ключами Data Protection. Підроблений або змінений токен не розшифрується, а прострочений буде відхилено;
- з токена відновлюється `ClaimsPrincipal` з клеймами (`NameIdentifier` = Id користувача, `Name`, `Email`, `Role`…) і записується в `HttpContext.User`;
- `AuthorizationMiddleware` перевіряє вимоги кінцевої точки, наприклад `User.IsInRole("Admin")`. Контролер потім бере користувача з `User` (`UserManager.GetUserAsync(User)` у `AccountController`);
- без заголовка або з недійсним токеном користувач анонімний, тож захищена кінцева точка повертає 401 і `WWW-Authenticate: Bearer`.

8. Які кінцеві точки REST API залишилися загальнодоступними, а які захищені — та чому саме такий розподіл? Наведіть приклади.

Загальнодоступні (`[AllowAnonymous]`): `GET /api/projects`, `GET /api/projects/{id}`, `GET /api/projects/{id}/store-page`, `/bugs`, `/team`, `GET /api/bugs`, `GET /api/bugs/{id}`, а також `/api/auth/register`, `/login`, `/refresh`. Це публічна інформація ігрової студії (каталог ігор, сторінки в магазині, відкритий баг-трекер), подібна до перегляду книг у бібліотеці. Реєстрація і вхід мають бути доступні до отримання токена.

Захищені: усі операції, що змінюють дані (POST, PUT, DELETE проєктів, багів, команд, сторінок у магазині), бо змінювати їх може лише відповідальна особа. Також дані працівників (`/api/programmers`, `/api/artists`) — це персональні дані з зарплатами, вони видимі лише ролям Staff. Профіль `/api/account/me` потребує токена, бо інакше незрозуміло, чий це профіль. `/api/users` — лише Admin. Приклад з розділу 3: `GET /api/projects` без токена — 200, `POST /api/projects` без токена — 401.

9. Як ви реалізували поділ повноважень між ролями (мінімум три ролі) та якими атрибутами (наприклад, [Authorize(Roles = "...")]) це забезпечується?

Ролі Admin, Manager, Developer, Tester створюються `IdentitySeeder` (таблиця `AspNetRoles`); роль Tester отримує кожен новий користувач, інші надає адміністратор через `PUT /api/users/{id}/roles/{role}`. Повноваження описано в розділі 1.4. Атрибути:

- `[Authorize]` на кожному контролері — будь-який автентифікований користувач (наприклад, `POST /api/bugs`, `/api/account/me`);
- `[AllowAnonymous]` — відкриті методи читання;

- `[Authorize(Roles = StudioRoles.Staff)]` ("Admin,Manager,Developer") — оновлення багів, перегляд працівників;
- `[Authorize(Roles = StudioRoles.AdminOrManager)]` — створення й зміна проєктів, команди, сторінки в магазині, видалення багів;
- `[Authorize(Roles = StudioRoles.Admin)]` — видалення проєкту, працівники (POST/PUT/DELETE), увесь `UsersController`.

Ролі через кому в одному атрибуті означають «будь-яка з них». Атрибути на класі й методі поєднуються: мають виконатися обидва. Назви ролей зібрано в константах `StudioRoles`, щоб не повторювати рядки. Ролі потрапляють у токен під час входу, тому зміна ролі діє після повторного входу або `/api/auth/refresh` (перевірено тестом `RefreshToken_ReturnsNewAccessTokenWithCurrentRoles`).

10. Як обробляються ситуації, коли користувачі неавторизовані або мають недостатні права? Які HTTP-коди повертаються в цих випадках?

Ситуація	Код	Відповідь
немає токена, токен недійсний або прострочений	**401 Unauthorized**	`WWW-Authenticate: Bearer`; ProblemDetails «Потрібна автентифікація» з підказкою, як отримати токен
токен дійсний, роль не дозволяє дію	**403 Forbidden**	ProblemDetails «Доступ заборонено. Роль користувача не дозволяє виконати цю дію.»
неправильний email або пароль при вході	**401 Unauthorized**	ProblemDetails від `MapIdentityApi`; після 5 невдалих спроб — тимчасове блокування
реєстрація з некоректними даними	**400 Bad Request**	`errors` з кодами Identity (`PasswordTooShort`, `DuplicateUserName`…)

401 і 403 формують обробники автентифікації (Challenge і Forbid) ще до виклику контролера, тому код контролерів не змінюється. Схема Bearer повертає код без перенаправлення на сторінку входу (як для cookie), що правильно для REST API. `UseStatusCodePages()` разом з `AddProblemDetails` додає до таких відповідей тіло у форматі `application/problem+json`. Swagger показує ці коди в документації захищених операцій.

11. У чому відмінність між автентифікацією та авторизацією, і як ці процеси взаємодіють у вашому застосунку?

Автентифікація відповідає на питання «хто ти?»: перевіряє облікові дані (email і пароль під час входу) або токен і встановлює особу користувача. **Авторизація** відповідає на питання «що тобі дозволено?»: за вже встановленою особою та її ролями вирішує, чи можна виконати дію. Помилка автентифікації — 401, помилка авторизації — 403.

У застосунку: `POST /api/auth/login` автентифікує користувача за паролем і видає токен. Далі в кожному запиті `UseAuthentication()` автентифікує його за токеном (заповнює `HttpContext.User` з клеймами ролей), а `UseAuthorization()` порівнює ролі з атрибутами `[Authorize(Roles = …)]`. Авторизація можлива лише після автентифікації: запит Tester до `POST /api/projects` проходить автентифікацію, але не авторизацію (403), а запит без токена не проходить автентифікацію (401).

12. Як ви тестували кінцеві точки реєстрації, автентифікації та авторизованих запитів? Які приклади потрібно включити в PDF-звіт?

- **автоматичні інтеграційні тести** (NUnit, `WebApplicationFactory<Program>`): 11 тестів `AuthTests` перевіряють реєстрацію, вхід, токени, оновлення токена і коди 200/201/204/400/401/403/404/409 для кожної ролі (розділ 4);
- **Swagger UI розгорнутого застосунку**: реєстрація, вхід, кнопка Authorize з токеном, запити від імені різних користувачів (розділ 3);
- перевірка вмісту: `GET /api/account/me` показує, яку роль сервер прочитав з токена; `GET /api/users` — що роль змінилася.

У PDF-звіт потрібно включити: реєстрацію (успішну), вхід з отриманими токенами (секрети замаскувати), запити без токена до відкритої (200) і захищеної (401) кінцевої точки, запити автентифікованого користувача, дозволені його роллю (200/201), запити з недостатньою роллю (403), запити користувача з вищою роллю до тих самих кінцевих точок (успіх), а також вигляд Swagger з позначенням захищених операцій.

13. Які кроки виконані для розгортання застосунку у вибраній хмарній/хостинговій платформі, і які сервіси були задіяні?

Платформа — **Render**. Кроки (детально в розділі 2):

- підготовка застосунку: `Dockerfile` (багатоетапне збирання .NET 8), `.dockerignore`, `render.yaml`; кінцева точка `/health`; обробка заголовків проксі (`UseForwardedHeaders`); Swagger у режимі Production; порт зі змінної `PORT`;
- відправлення коду в гілку `lab5` публічного репозиторію GitHub;
- створення Web Service у Render з цього репозиторію: гілка `lab5`, середовище Docker, регіон Frankfurt, план Free, Health Check Path `/health`;
- додавання змінних середовища (`Seed__AdminEmail`, секрет `Seed__AdminPassword`);
- автоматичне збирання образу й запуск; перевірка журналу, `/health` і запитів через Swagger.

Задіяні сервіси: **Render Web Services** (збирання Docker-образу, запуск контейнера, перевірка стану, журнали, метрики, автоматичне розгортання після push), керований TLS-сертифікат і домен `onrender.com`, мережа доставки Cloudflare перед сервісом (заголовки `cf-ray`, `server: cloudflare`), **GitHub** як джерело коду, базові образи Microsoft .NET з реєстру `mcr.microsoft.com`.

14. Як ви налаштували перенос налаштувань (connection string, JWT-секрети тощо) у середовище розгортання, та як забезпечили безпеку цих даних?

ASP.NET Core читає конфігурацію шарами: `appsettings.json` → `appsettings.{Environment}.json` → змінні середовища → аргументи командного рядка. Пізніший шар перекриває попередній, а `__` у назві змінної відповідає `:` (`Seed__AdminPassword` → `Seed:AdminPassword`). Тому код не залежить від середовища:

- **рядок підключення** не є секретом (шлях до файлу SQLite в контейнері), тож задано в `Dockerfile`: `ConnectionStrings__GameStudio="Data`

`Source=/app/Database/gamestudio.db"`; там само `ASPNETCORE_ENVIRONMENT=Production`;

- **пароль адміністратора** — секрет. У репозиторії його немає: в `appsettings.json` значення порожнє, у `render.yaml` — `sync: false`. Значення введено вручну в розділі Environment панелі Render. Render зберігає його в налаштуваннях сервісу поза репозиторієм, показує замаскованим і передає в контейнер лише як змінну середовища. Для локального запуску є окремий пароль в `appsettings.Development.json`, який на сервері (середовище Production) не завантажується;
- **ключі підпису токенів**: токени Bearer Identity захищено ключами Data Protection, які генеруються в контейнері й не потрапляють у репозиторій. Якщо використовувати JWT, секретний ключ задавався б так само змінною середовища (наприклад, `Jwt__Key`) і ніколи не записувався б у код;
- увесь трафік іде через HTTPS (сертифікат Render), тому токени й паролі не передаються відкритим текстом; контейнер працює від непривілейованого користувача `app`.

Недолік безкоштовного плану: ключі Data Protection і файл БД зберігаються лише в тимчасовій файловій системі, тому після перезапуску видані токени стають недійсними (у журналі є попередження `No XML encryptor configured`). Для робочого середовища ключі варто зберігати в постійному сховищі (`PersistKeysToDbContext` або Azure Key Vault), а дані — у керованій БД.

15. Які перевірки виконувалися, щоб упевнитися, що розгорнута версія застосунку працює коректно, та які матеріали (посилання, скріншоти) потрібно додати до звіту?

- статус розгортання в Render — Deploy succeeded / Live; перевірка стану `/health` повертає 200 (без неї Render не перемкнув би трафік на нову версію);
- журнал застосунку: міграції застосовано, створено ролі та адміністратора, Kestrel слухає порт 10000, середовище Production, помилок немає;
- Swagger UI відкривається за адресою застосунку, а корінь `/` перенаправляє на нього; у заголовках відповідей — `x-render-origin-server: Kestrel`;
- функціональна перевірка: сценарій із 16 запитів (розділ 3) — анонімний доступ, реєстрація, вхід, запити Tester і Admin; усі коди відповідей збіглися з очікуваними й з результатами автоматичних тестів, що виконуються локально перед кожним push;
- перевірка безпеки: захищені кінцеві точки без токена повертають 401, з недостатньою роллю — 403; секрет не видно ні в репозиторії, ні в панелі Render.

До звіту додано: посилання на застосунок (https://gamestudio-api-ni3x.onrender.com) і на Swagger UI, скріншоти сервісу, розгортання, налаштувань збирання, змінних середовища (значення приховано) та журналу (розділ 2.5), а також скріншоти запитів до розгорнутого застосунку з адресою onrender.com у полі Request URL (розділ 3).